연합트윈 기술해설서

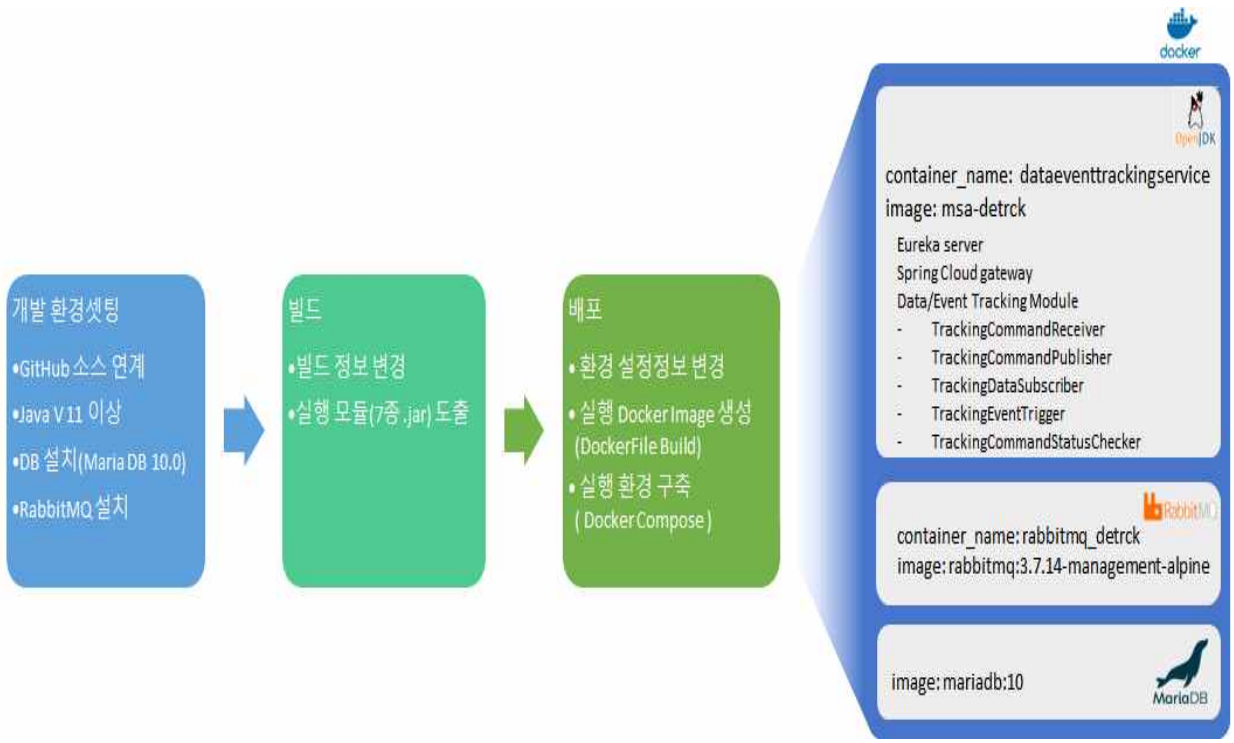
동적 데이터 – 이벤트 트래킹

<목차>

1. 개요 ····· 3
가. Data/Event Tracking 제작 프로세스 ····· 3
2. 개발 환경 구성 (Windows 기반) ····· 5
가. 개발 환경 정보 ····· 5
나. Spring Boot 설치 ····· 5
다. Lombok 설치 ····· 6
라. 동적 데이터/이벤트 트래킹 설치 ····· 8
마. DB 세팅 ····· 12
바. RabbitMQ 세팅 ····· 18
3. 모듈 별 기능 구성 및 설정 정보 ····· 23
가. 동적 데이터/이벤트 트래킹 S/W구성 및 기능 구성 ····· 23
4. 구동/실행 ····· 31
가. Data/Event Tracking Local 구동/실행 ····· 31
5. 동적 데이터/이벤트 트래킹 배포 ····· 36
가. 서버 배포 구성 ····· 36
6. 동적 데이터/이벤트 트래킹 인터페이스 ····· 45
가. 인터페이스 ····· 45
7. Data/Event Tracking Command 정의 ····· 50
가. Data/Event Tracking Process Info 정의 ····· 50
나. Data Tracking 연계 값 설정(trckValInfo) ····· 52
다. 데이터/이벤트 트래킹 예시 ····· 55

1. 개요

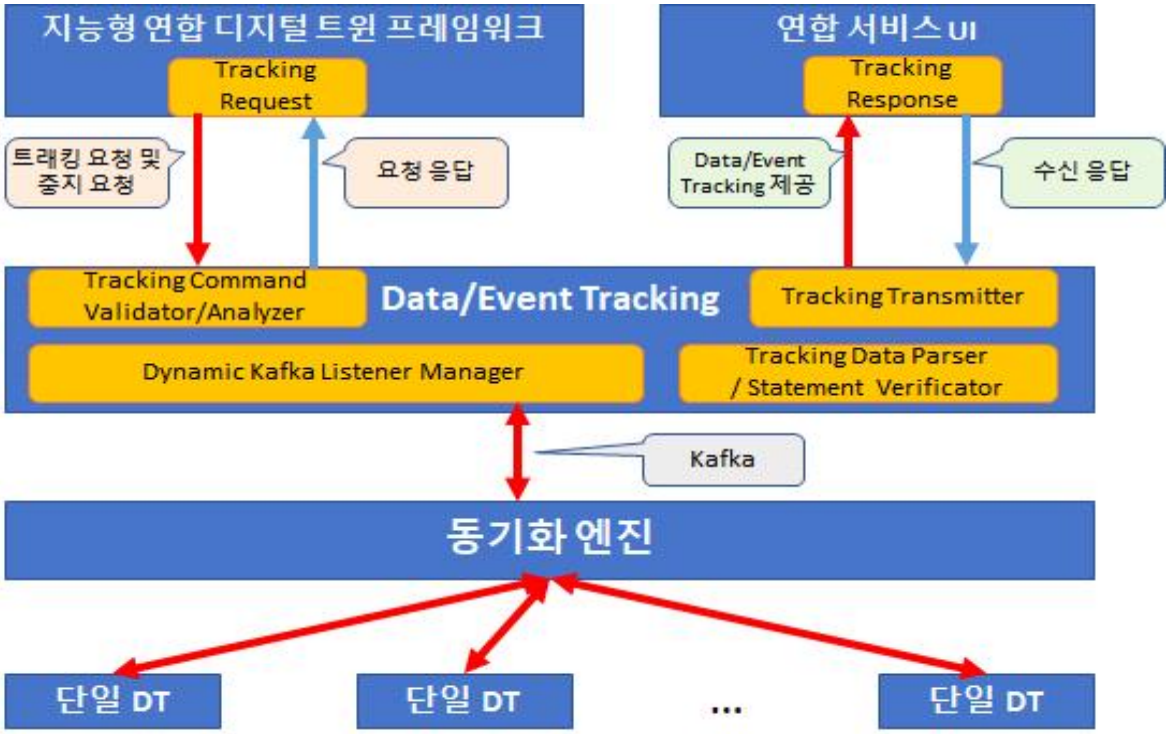
가. Data/Event Tracking 제작 프로세스



< Data/Event Tracking 제작 프로세스 >

Data/Event Tracking은 개발부터 배포까지 총 3단계(개발 환경 셋팅 → 빌드 → 배포)의 과정을 통해 운용 환경을 구축할 수 있습니다. 이 시스템은 docker compose 기반으로 배포되며, docker compose build/up 실행 시 실행/환경파일(com.env, env.prod 및 docker-compose.yml)을 운용 환경에 맞게 설정함으로써 유연한 시스템 구축이 가능합니다.

운용 환경이 구축(배포)된 이후에는, 연합 또는 단일 Digital Twin(DT)과의 데이터 연동을 위해 Data/Event Tracking Command를 생성하거나 요청해야 합니다. 이 Command는 사용자 설정정보 및 고속・고신뢰 양방향 동기화 엔진이 제공하는 연합/단일 DT의 메타정보 및 데이터 구조를 기반으로 하여 동적으로 생성할 수 있으며 Data/Event Tracking Command를 통하여 다양한 형태의 DT 간 데이터를 효과적으로 통제하고 추적할 수 있습니다.



또한, 해당 시스템을 활용하고자 하는 외부 서비스에서는 Data/Event Tracking 수신용 CallBack 기능을 반드시 구현해야 하며 이 수신 정보(CallBack URL)는 Command 실행 시 필수 요소로 조합되며, Command가 정상적으로 수행되기 위해 반드시 필요합니다.

각 프로세스 별(개발 환경셋팅 -> 빌드 -> 배포 -> 명령 실행) 설명은 아래에 연이어 기술하도록 하겠습니다.

2. 개발 환경 구성 (Windows 기반)

가. 개발 환경 정보

1) 언어 : Java 11

2) 프레임워크 : Spring Boot 4.30.0

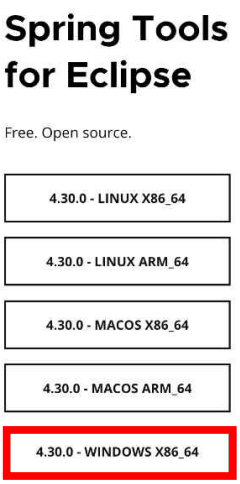
3) DB : MariaDB 10.0

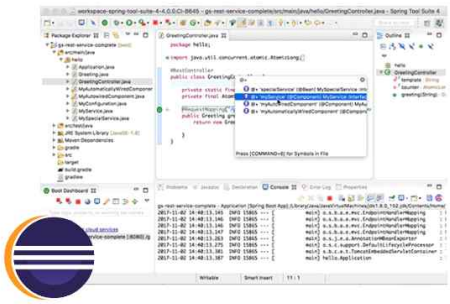
4) 프로젝트 구조 : Spring Boot Maven Multi Module 구조

나. Spring Boot 설치

1) STS4 4.30.0 설치

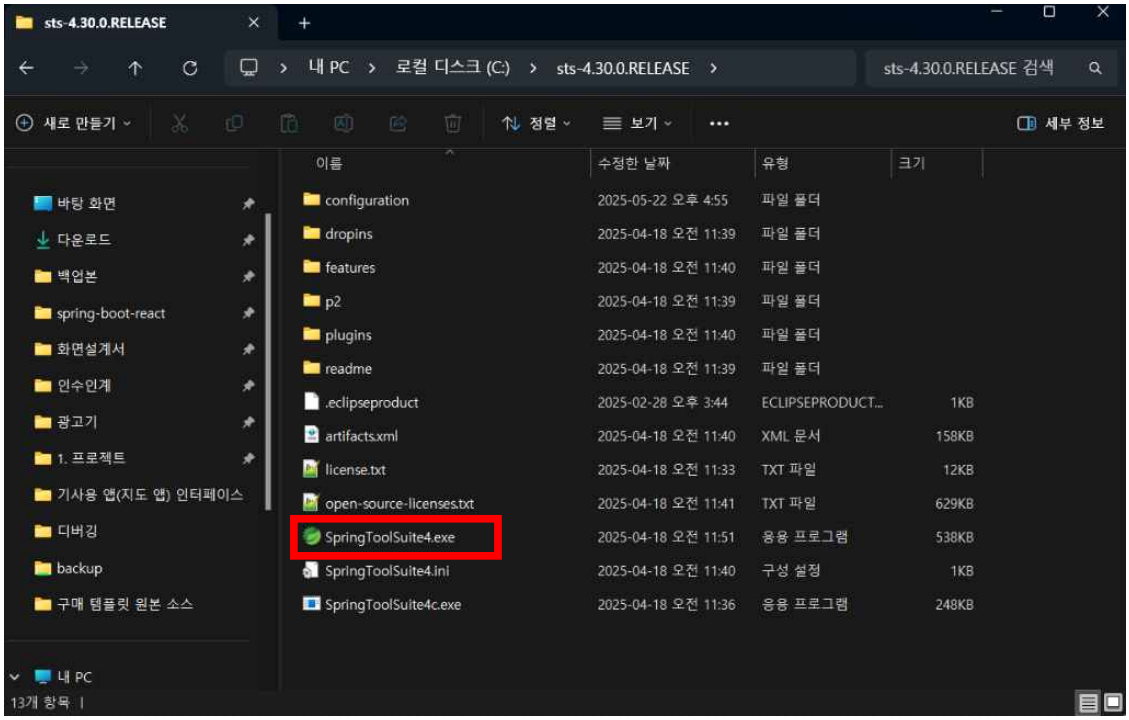
- https://spring.io/tools





2) 다운받은 zip 파일 압축 해제 후 원하는 경로로 파일 이동

3) 파일 속 SpringToolSuite4.exe 클릭

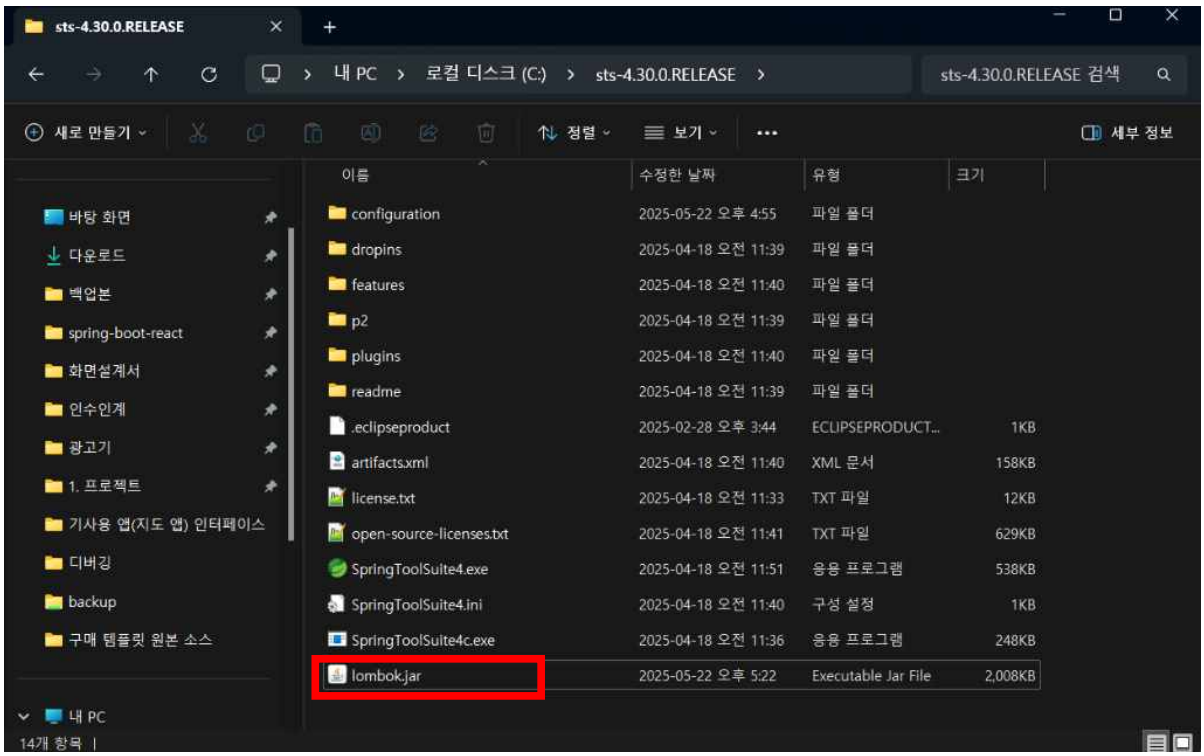


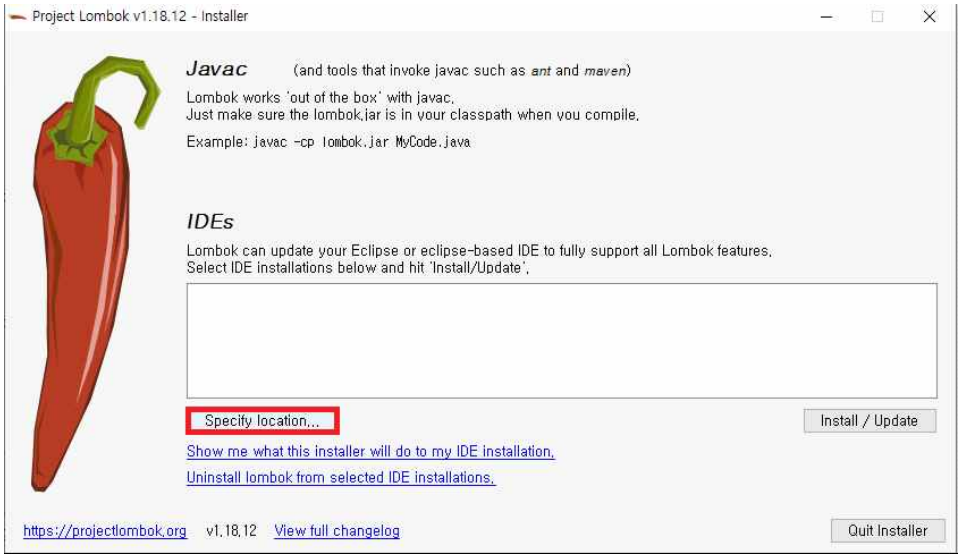
다. Lombok 설치

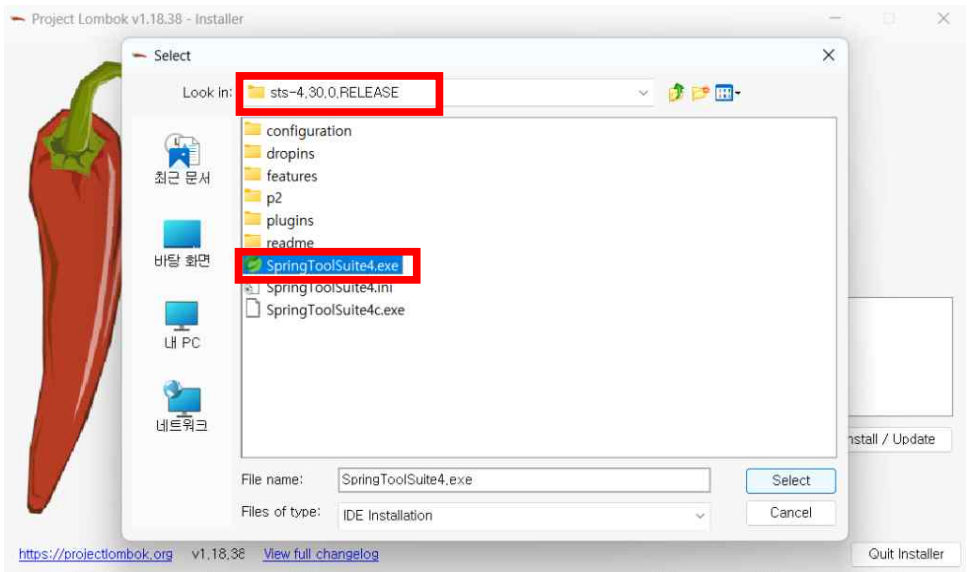
1) 롬복 설치

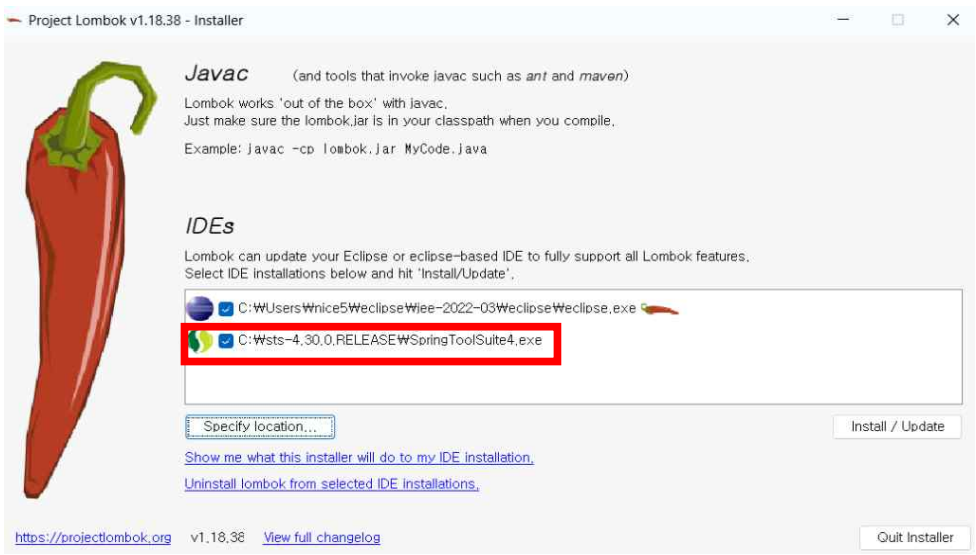
- https://projectlombok.org/download

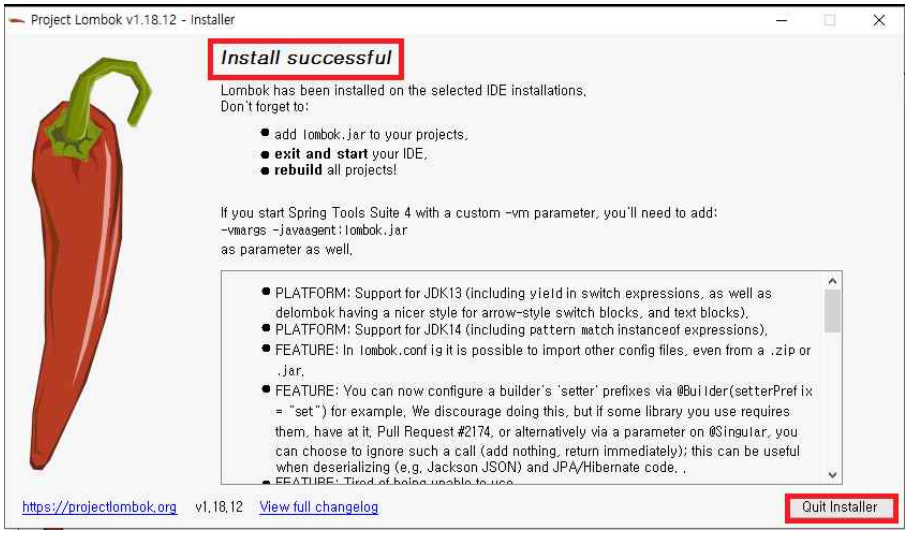
2) STS가 설치된 경로에 lombok.jar를 이동시킨 후 실행




Project Lombok v1.18.12 - Installer
Javac
(and tools that invoke javac such as ant and maven)
Lombok works 'out of the box' with javac.
Just make sure the lombok.jar is in your classpath when you compile.
Example: javac -cp lombok.jar MyCode.java
IDEs
Lombok can update your Eclipse or eclipse-based IDE to fully support all Lombok features.
Select IDE installations below and hit 'Install/Update'.
Specify location...
Install / Update
Show me what this installer will do to my IDE installation.
Uninstall lombok from selected IDE installations.
https://projectlombok.org
v1.18.12
View full changelog
Quit Installer

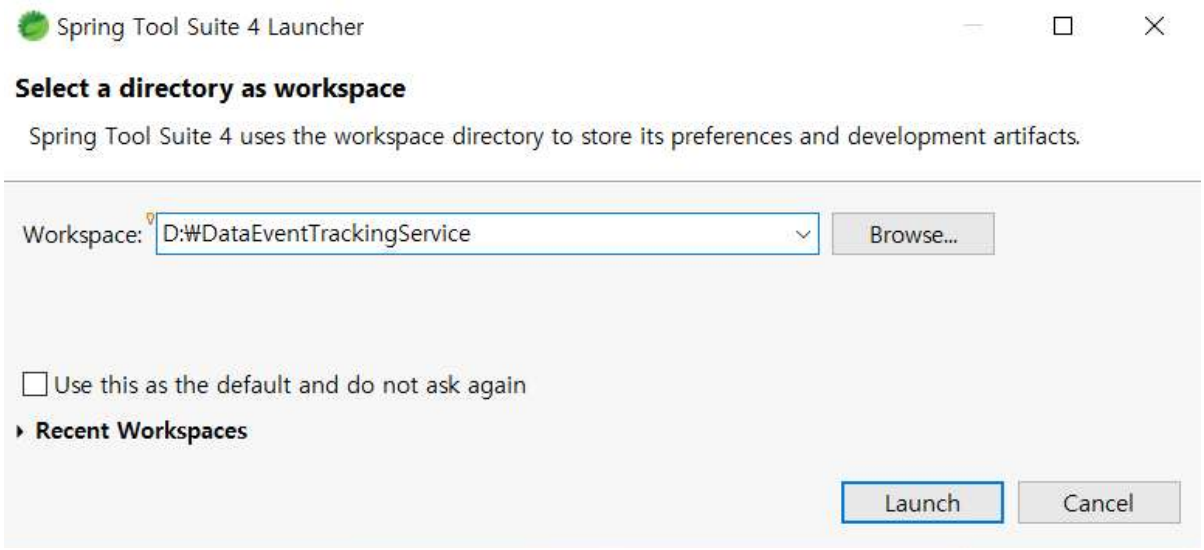

Project Lombok v1.18.38 - Installer
Select
Look in:
sts-4.30.0.RELEASE
configuration
dropins
features
p2
plugins
readme
SpringToolSuite4.exe
SpringToolSuite4.ini
SpringToolSuite4c.exe
최근 문서
바탕 화면
내 PC
네트워크
File name:
SpringToolSuite4.exe
Select
Files of type:
IDE Installation
Cancel
https://projectlombok.org
v1.18.38
View full changelog
Quit Installer


Project Lombok v1.18.38 - Installer
Javac
(and tools that invoke javac such as ant and maven)
Lombok works 'out of the box' with javac.
Just make sure the lombok.jar is in your classpath when you compile.
Example: javac -cp lombok.jar MyCode.java
IDEs
Lombok can update your Eclipse or eclipse-based IDE to fully support all Lombok features.
Select IDE installations below and hit 'Install/Update'.
C:₩Users₩nice5₩eclipse₩iee-2022-03₩eclipse₩eclipse.exe
C:₩sts-4.30.0.RELEASE₩SpringToolSuite4.exe
Specify location...
Install / Update
Show me what this installer will do to my IDE installation.
Uninstall lombok from selected IDE installations.
https://projectlombok.org
v1.18.38
View full changelog
Quit Installer

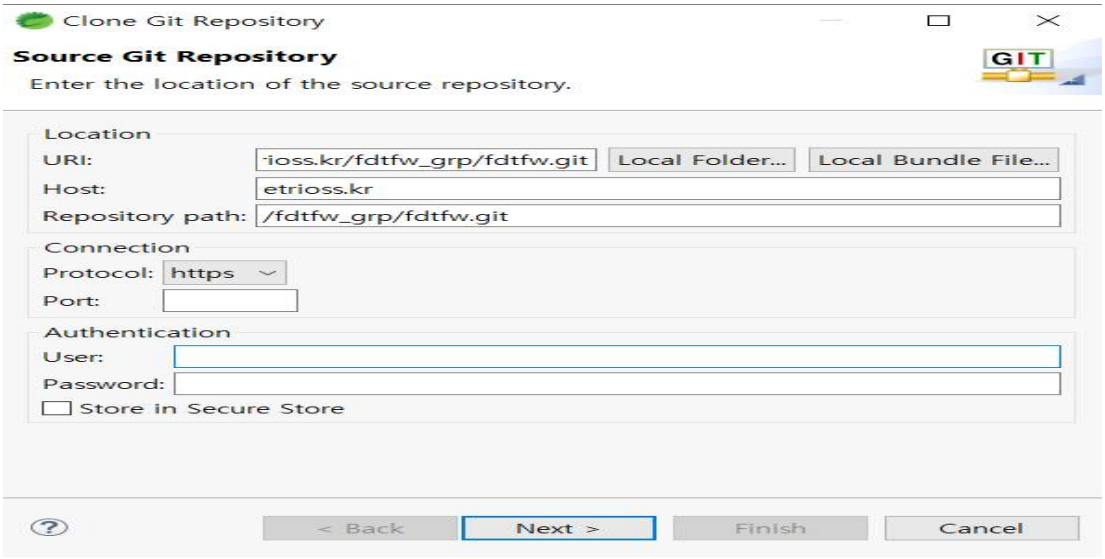


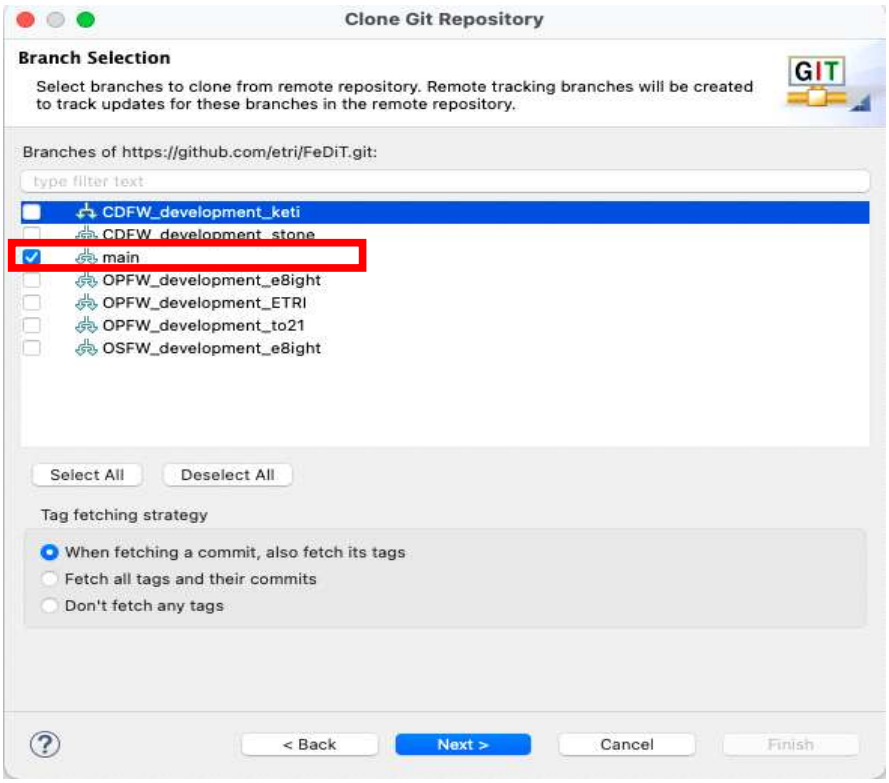
라. 동적 데이터/이벤트 트래킹 설치

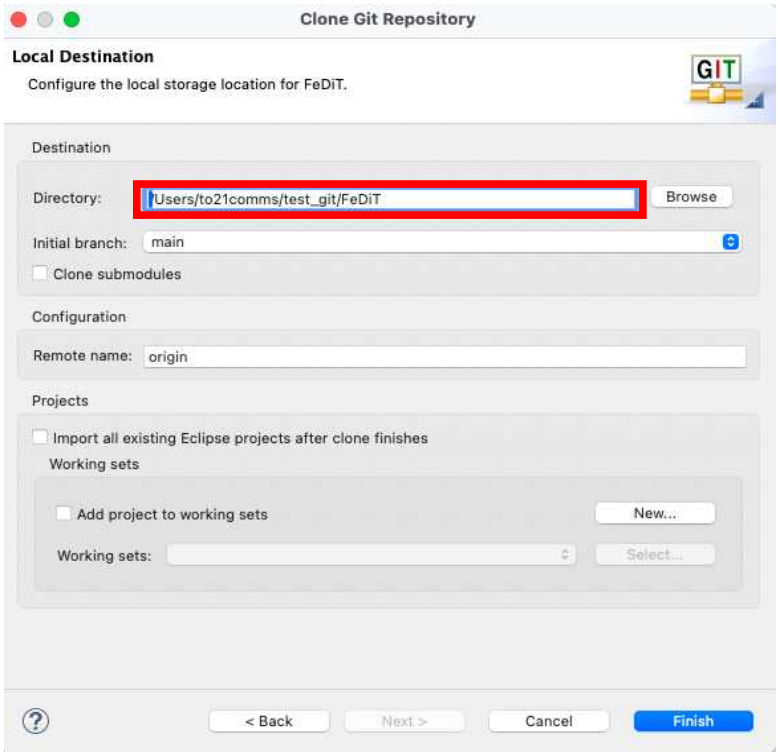
1) STS 접속 후 원하는 경로에 WorkSpace 지정



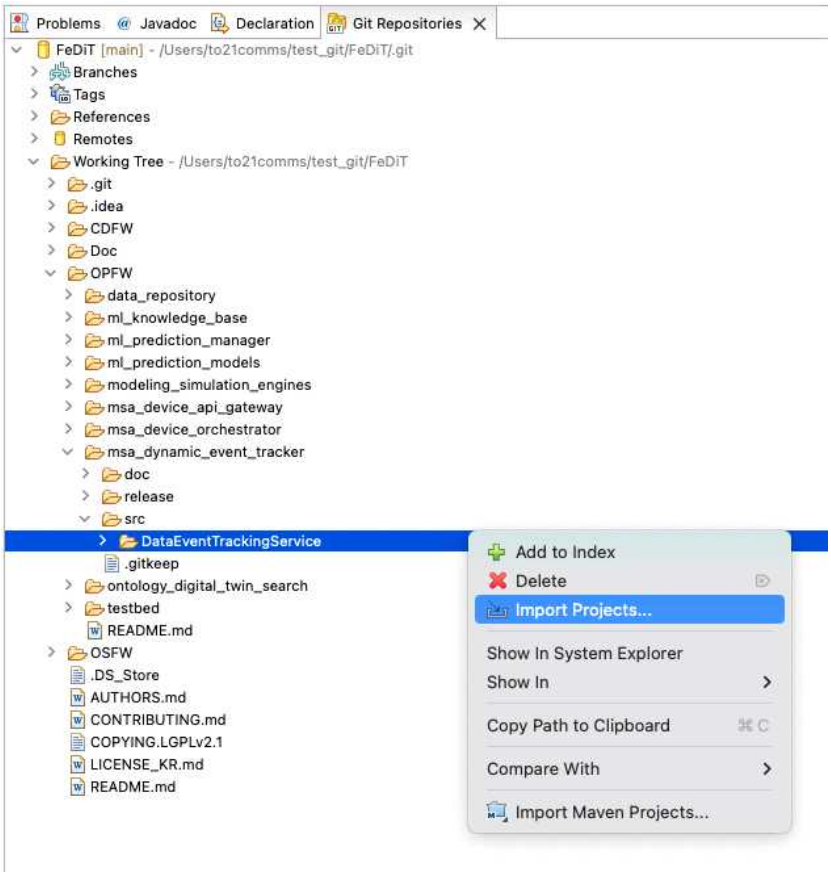
2) Window - Show View - Other - Git - Git Repositories 선택
3) Clone a Git repository 클릭
4) URI 영역에 https://github.com/etri/FeDiT.git 붙여넣기 - Git 계정정보 입력 - Next - main branch 선택

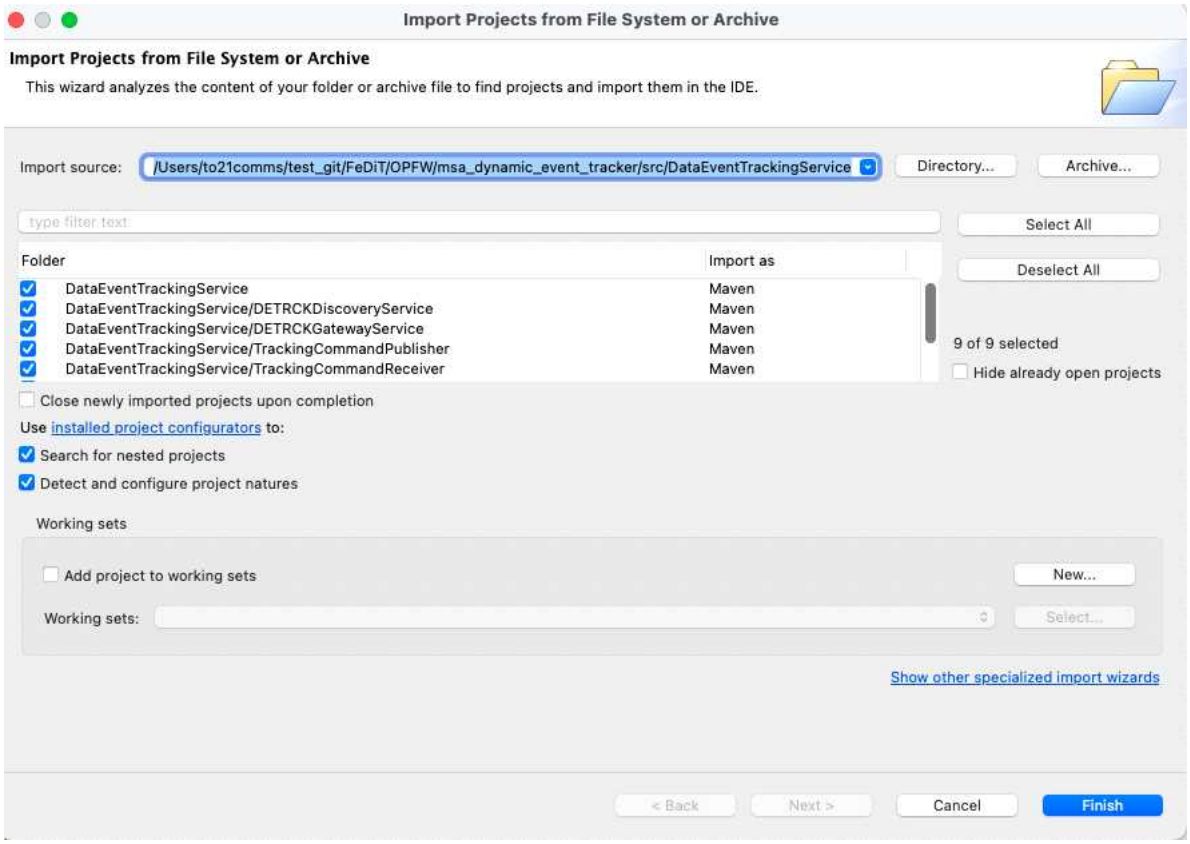

Clone Git Repository
Source Git Repository
Enter the location of the source repository.
GIT
Location
URI:
:ioss.kr/fdtfw_grp/fdtfw.git
Local Folder...
Local Bundle File...
Host:
etrioss.kr
Repository path:
/fdtfw_grp/fdtfw.git
Connection
Protocol:
https
Port:
Authentication
User:
Password:
Store in Secure Store
< Back
Next >
Finish
Cancel

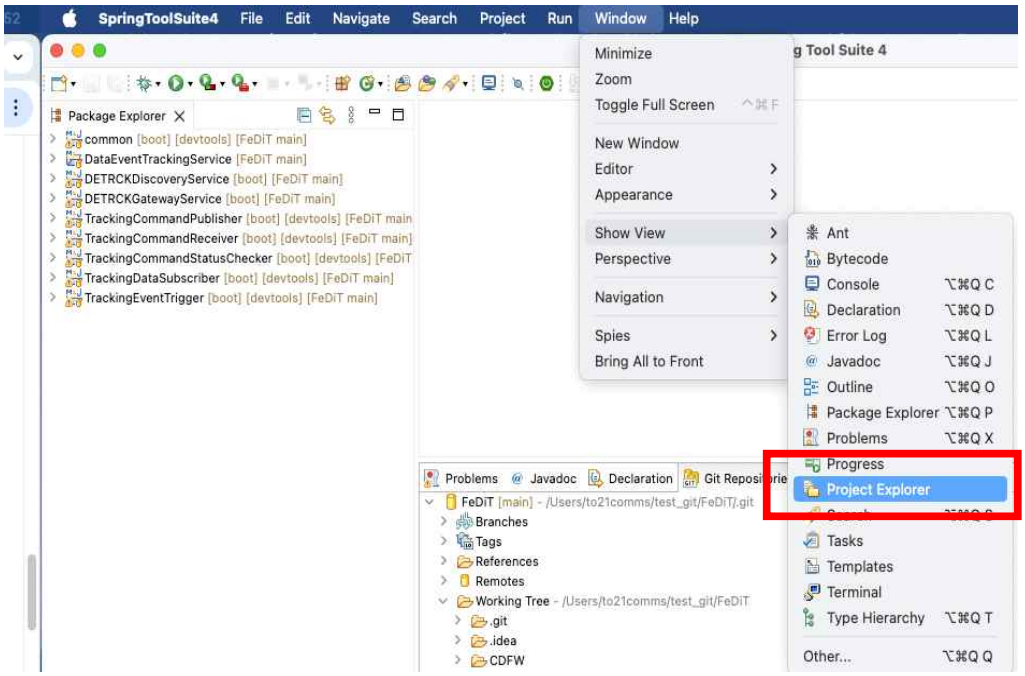

Clone Git Repository
Branch Selection
Select branches to clone from remote repository. Remote tracking branches will be created to track updates for these branches in the remote repository.
GIT
Branches of https://github.com/etri/FeDiT.git:
type filter text
CDFW_development_keti
CDFW_development_stone
main
OPFW_development_e8ight
OPFW_development_ETRI
OPFW_development_to21
OSFW_development_e8ight
Select All
Deselect All
Tag fetching strategy
When fetching a commit, also fetch its tags
Fetch all tags and their commits
Don't fetch any tags
< Back
Next >
Cancel
Finish

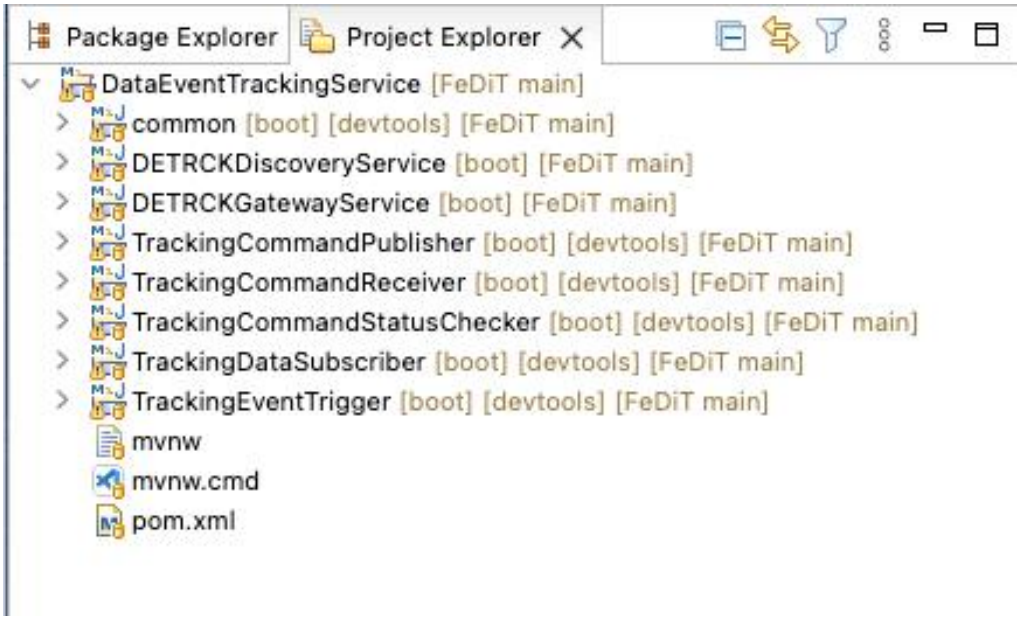


5) 하단 Git Repositories - Working Tree - OPFW - msa_dynamic_event_tracker- src - DataEventTrackingService 마우스 우클릭 - Import Projects




Import Projects from File System or Archive
Import Projects from File System or Archive
This wizard analyzes the content of your folder or archive file to find projects and import them in the IDE.
Import source:
/Users/to21comms/test_git/FeDiT/OPFW/msa_dynamic_event_tracker/src/DataEventTrackingService
Directory...
Archive...
type filter text
Select All
Folder
Import as
Deselect All
DataEventTrackingService
Maven
DataEventTrackingService/DETRCKDiscoveryService
Maven
DataEventTrackingService/DETRCKGatewayService
Maven
DataEventTrackingService/TrackingCommandPublisher
Maven
DataEventTrackingService/TrackingCommandReceiver
Maven
9 of 9 selected
Hide already open projects
Close newly imported projects upon completion
Use installed project configurators to:
Search for nested projects
Detect and configure project natures
Working sets
Add project to working sets
New...
Working sets:
Select...
Show other specialized import wizards
< Back
Next >
Cancel
Finish


SpringToolSuite4
File
Edit
Navigate
Search
Project
Run
Window
Help
Minimize
Zoom
Toggle Full Screen
New Window
Editor
Appearance
Show View
Perspective
Navigation
Spies
Bring All to Front
Ant
Bytecode
Console
Declaration
Error Log
Javadoc
Outline
Package Explorer
Problems
Progress
Project Explorer
Tasks
Templates
Terminal
Type Hierarchy
Other...
Package Explorer
common [boot] [devtools] [FeDiT main]
DataEventTrackingService [FeDiT main]
DETRCKDiscoveryService [boot] [FeDiT main]
DETRCKGatewayService [boot] [FeDiT main]
TrackingCommandPublisher [boot] [devtools] [FeDiT main]
TrackingCommandReceiver [boot] [devtools] [FeDiT main]
TrackingCommandStatusChecker [boot] [devtools] [FeDiT]
TrackingDataSubscriber [boot] [devtools] [FeDiT main]
TrackingEventTrigger [boot] [devtools] [FeDiT main]
Problems
Javadoc
Declaration
FeDiT [main] - /Users/to21comms/test_git/FeDiT/.git
Branches
Tags
References
Remotes
Working Tree - /Users/to21comms/test_git/FeDiT
.git
.idea
CDFW

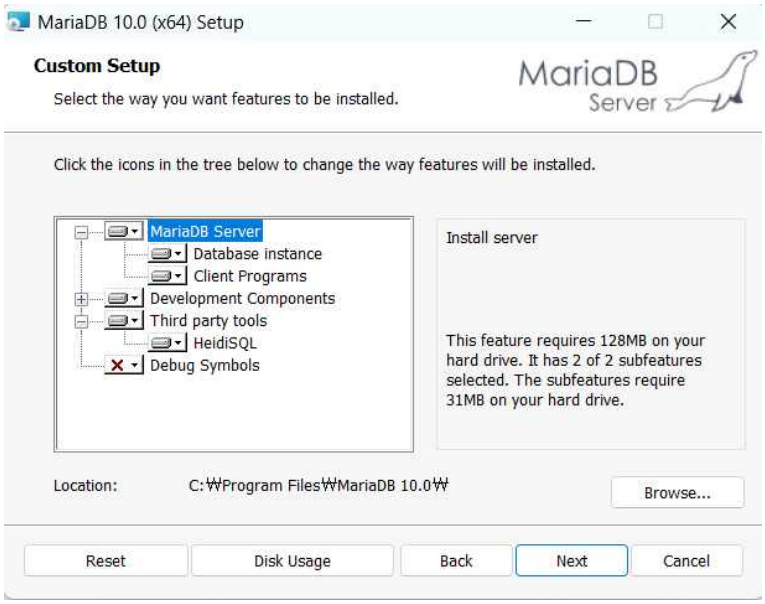


마. DB 세팅

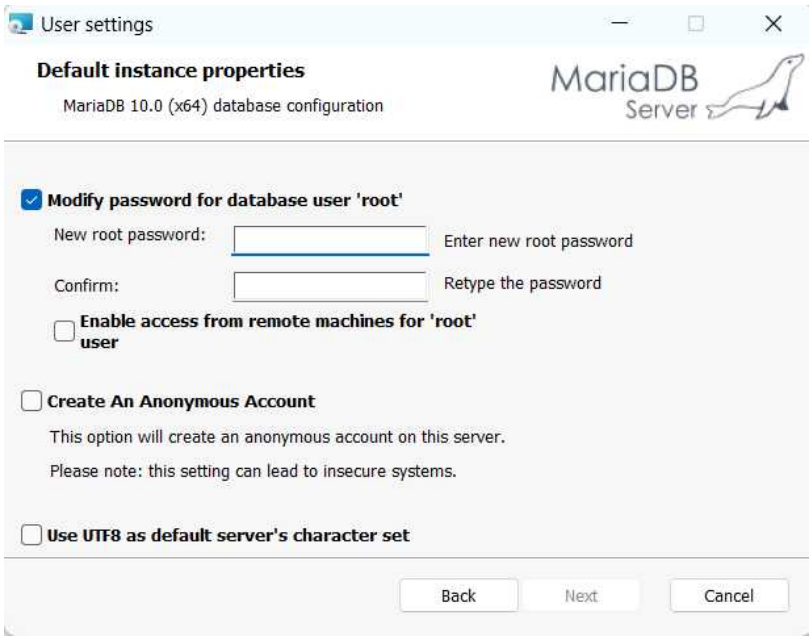
1) MariaDB 10.0 설치

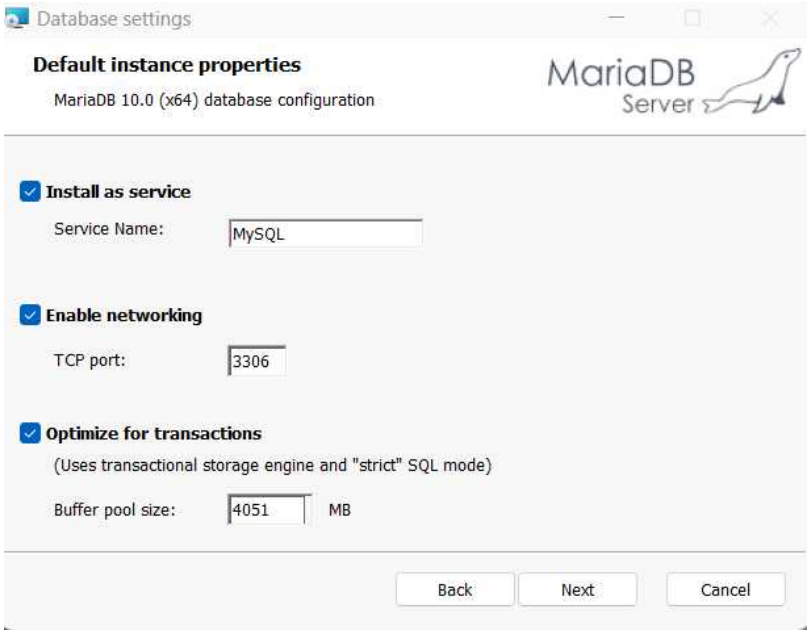
- https://archive.mariadb.org/mariadb-10.0.38/winx64-packages/mariadb-10.0.38-winx64.msi



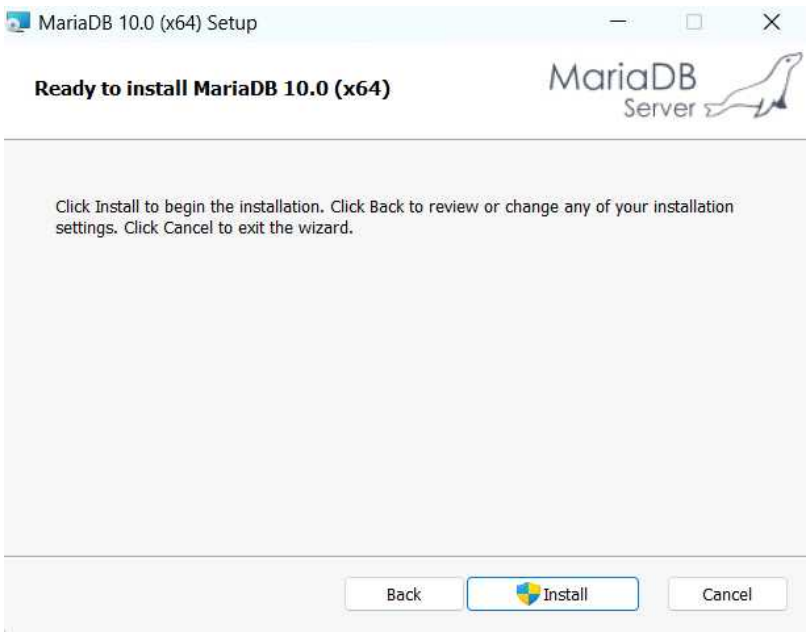


- root 계정 비밀번호 입력





- TCP port 3306 중복 시 다른 포트로 변경

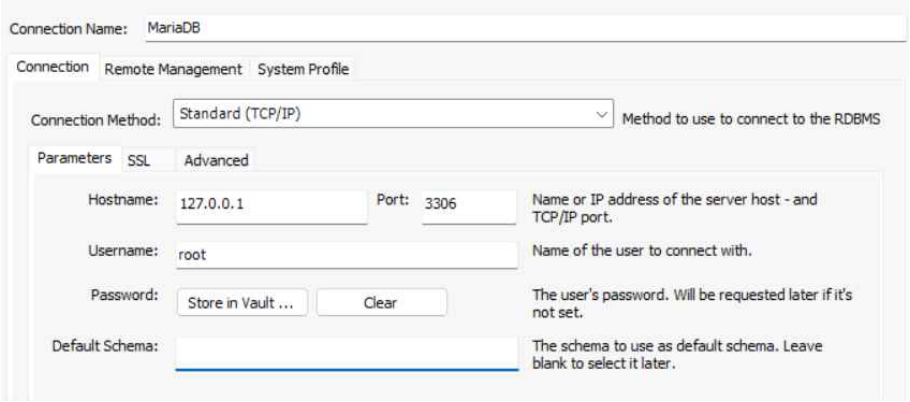


2) MariaDB 접속

- MariaDB는 공식 UI툴이 제공되지 않아 GUI 툴을 사용하여 접속
(e.g. Mysql workbench, DBeaver, HeidiSQL)

3) MariaDB 설치 중 생성한 root 계정과 비밀번호 입력





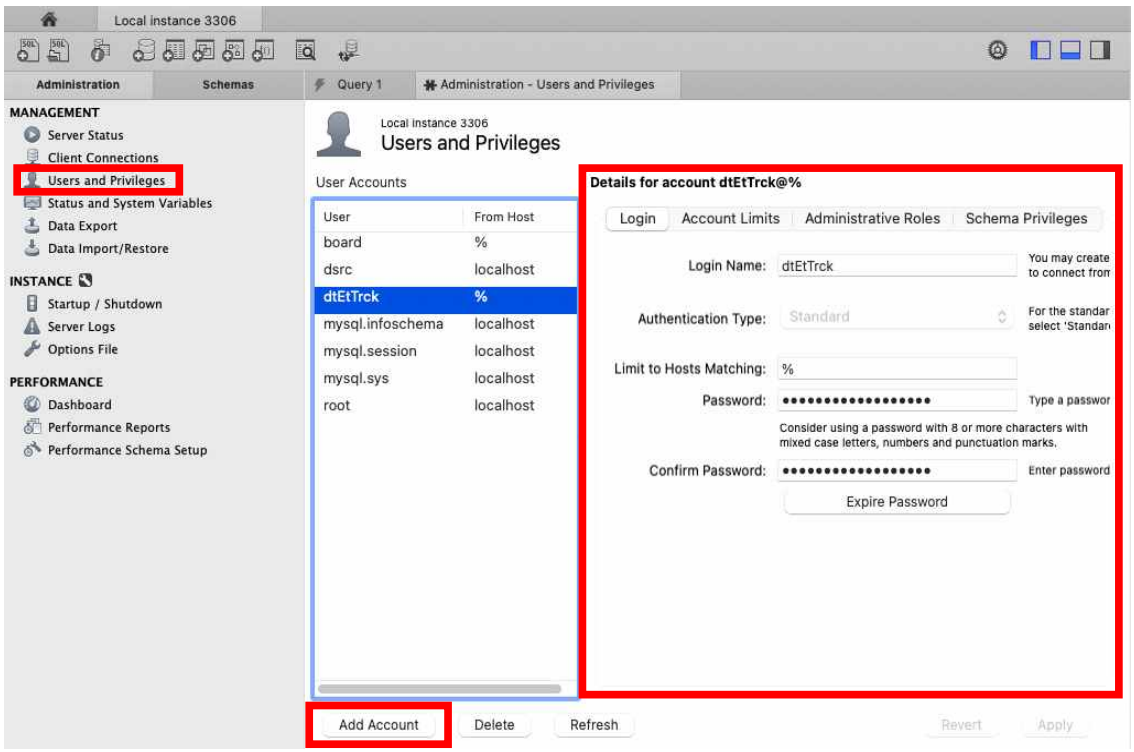
(a) 좌측 탭 Administration - Users and Privileges 클릭

(b) 하단의 Add Account 클릭 하여 사용자 추가

(c) 계정 정보 입력 후 Apply

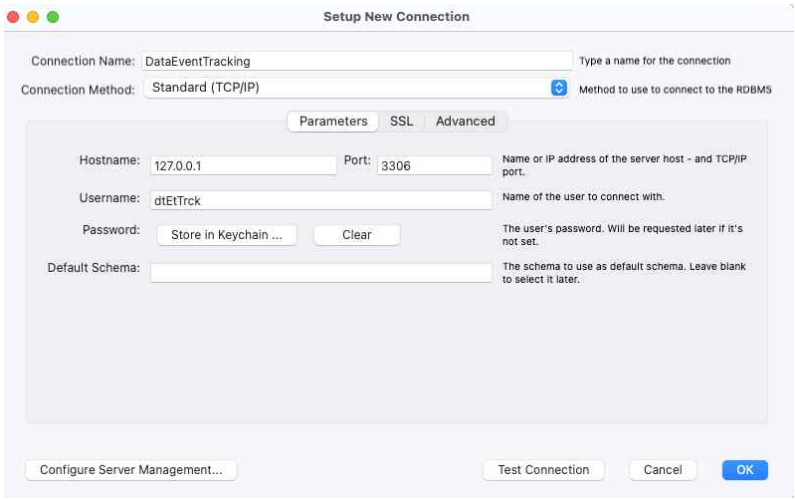
- 기본 계정 정보 생성 ID : dtEtTrck / PW : Fdt1234!@#$

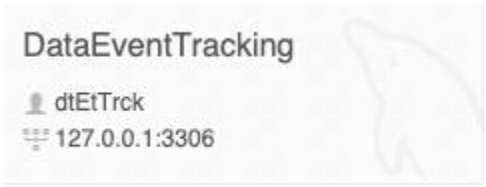
(계정 정보 수정 시 apprication.propertise에 jdbc 접속 정보 수정)



(d) dtEtTrck 계정으로 DB Connection

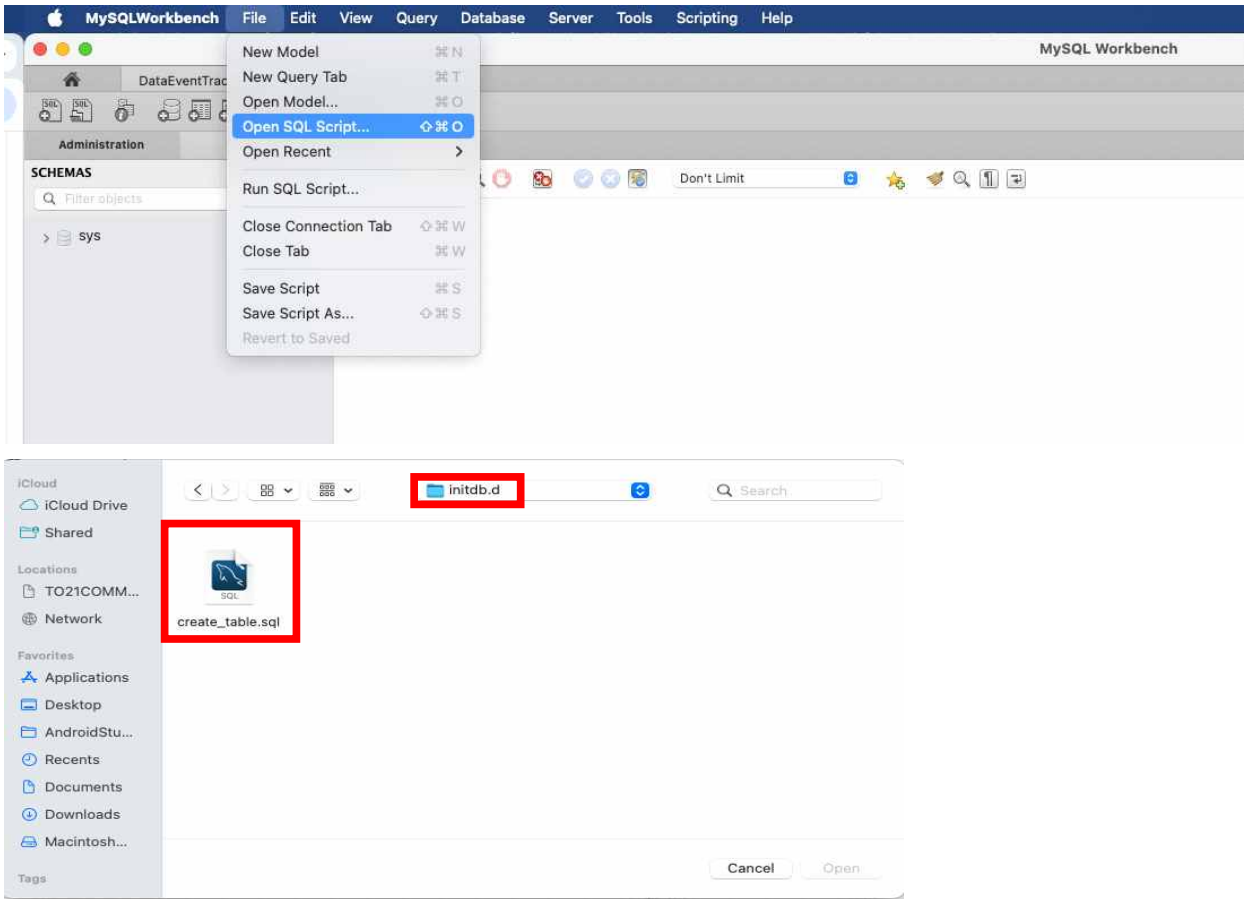


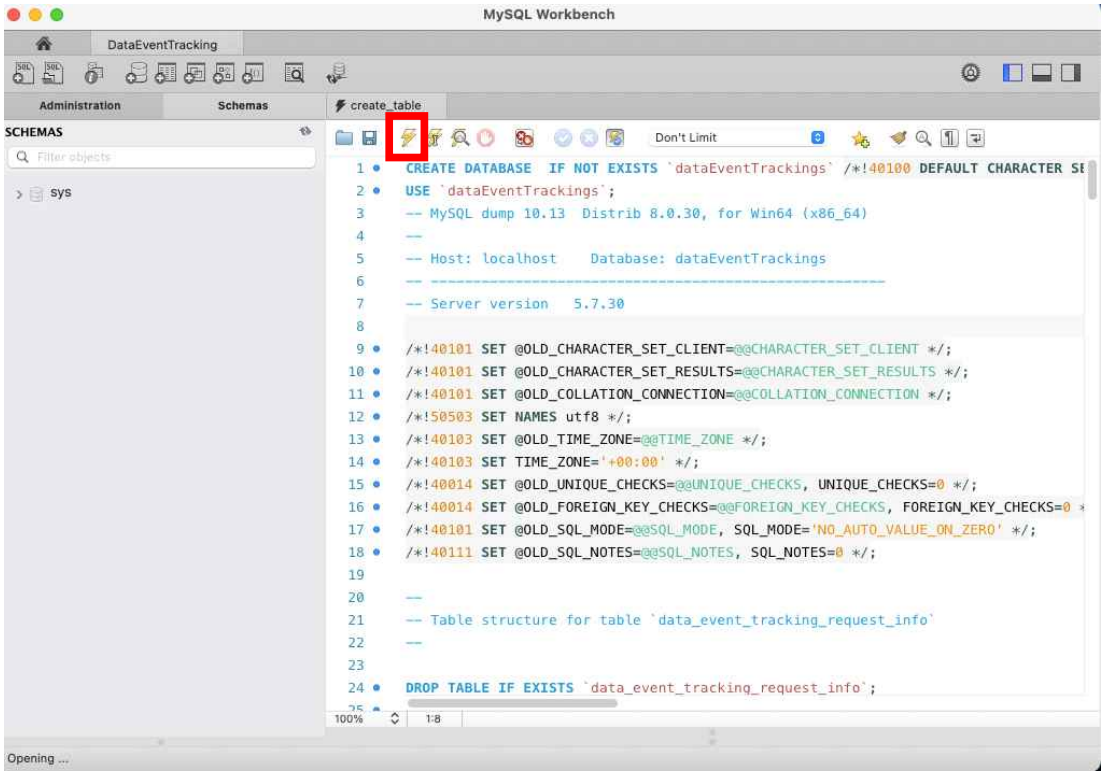


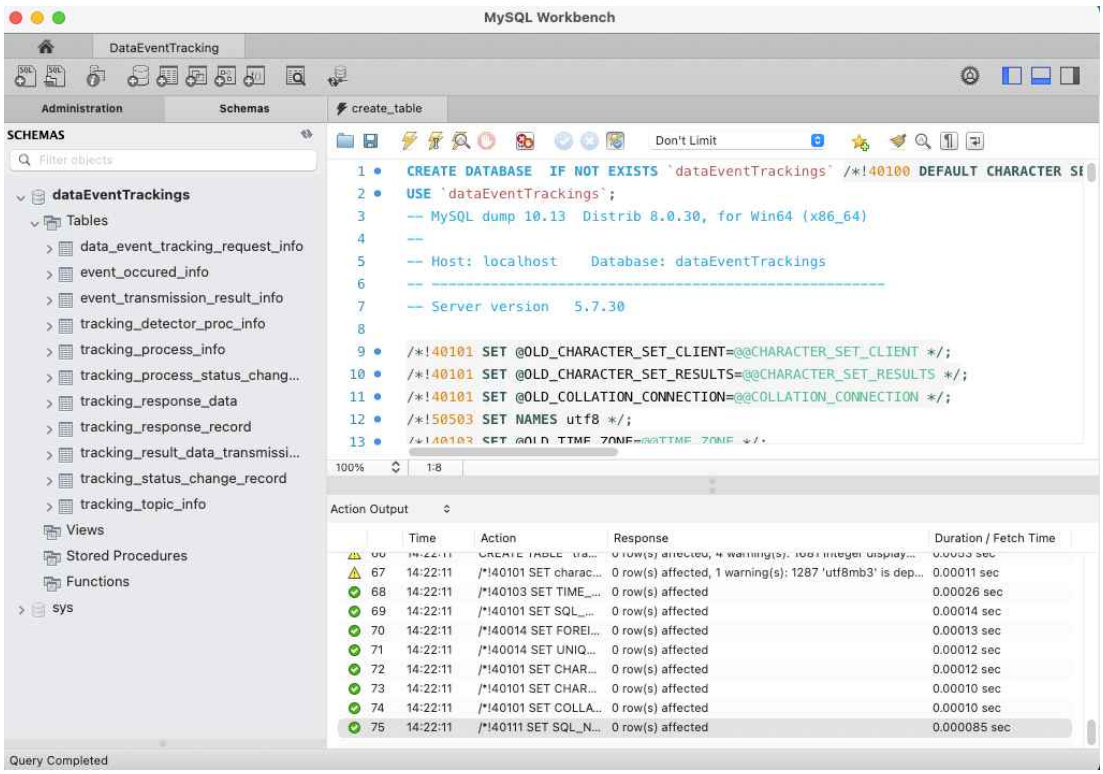


4) Open SQL Script 선택 후 Script 불러오기

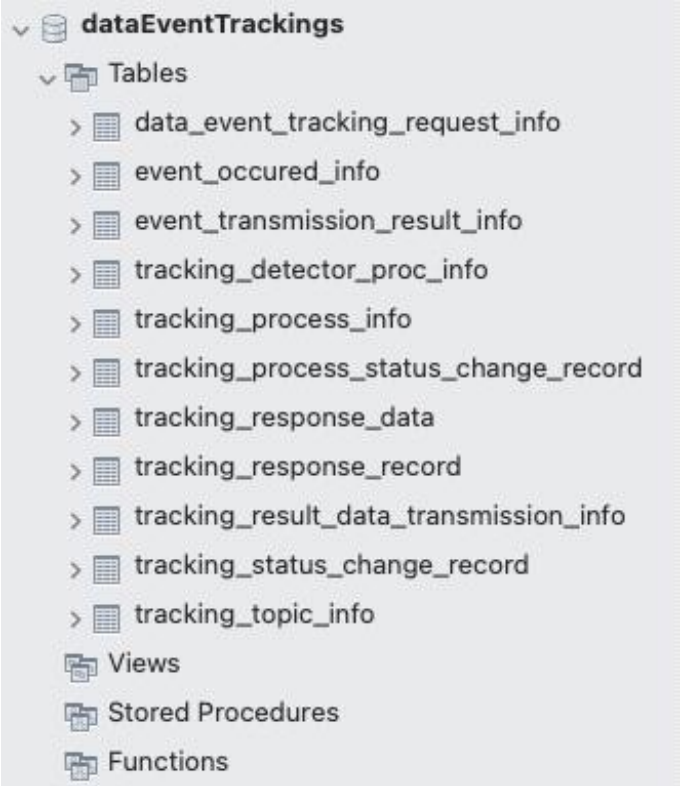
- github **initResource/DataEventTracingService/db/initdb.d** 폴더 속 create_table.sql 파일 저장 후 불러오기




MySQL Workbench
DataEventTracking
Administration
Schemas
create_table
SCHEMAS
Filter objects
sys
Don't Limit
1 CREATE DATABASE IF NOT EXISTS `dataEventTrackings` /*!40100 DEFAULT CHARACTER SE
2 USE `dataEventTrackings`;
3 -- MySQL dump 10.13 Distrib 8.0.30, for Win64 (x86_64)
4 --
5 -- Host: localhost Database: dataEventTrackings
6 -- --
7 -- Server version 5.7.30
8
9 /*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
10 /*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
11 /*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
12 /*!50503 SET NAMES utf8 */;
13 /*!40103 SET @OLD_TIME_ZONE=@@TIME_ZONE */;
14 /*!40103 SET TIME_ZONE='+00:00' */;
15 /*!40014 SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0 */;
16 /*!40014 SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0
17 /*!40101 SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='NO_AUTO_VALUE_ON_ZERO' */;
18 /*!40111 SET @OLD_SQL_NOTES=@@SQL_NOTES, SQL_NOTES=0 */;
19
20 --
21 -- Table structure for table `data_event_tracking_request_info`
22 --
23
24 DROP TABLE IF EXISTS `data_event_tracking_request_info`;
100%
1:8
Opening ...

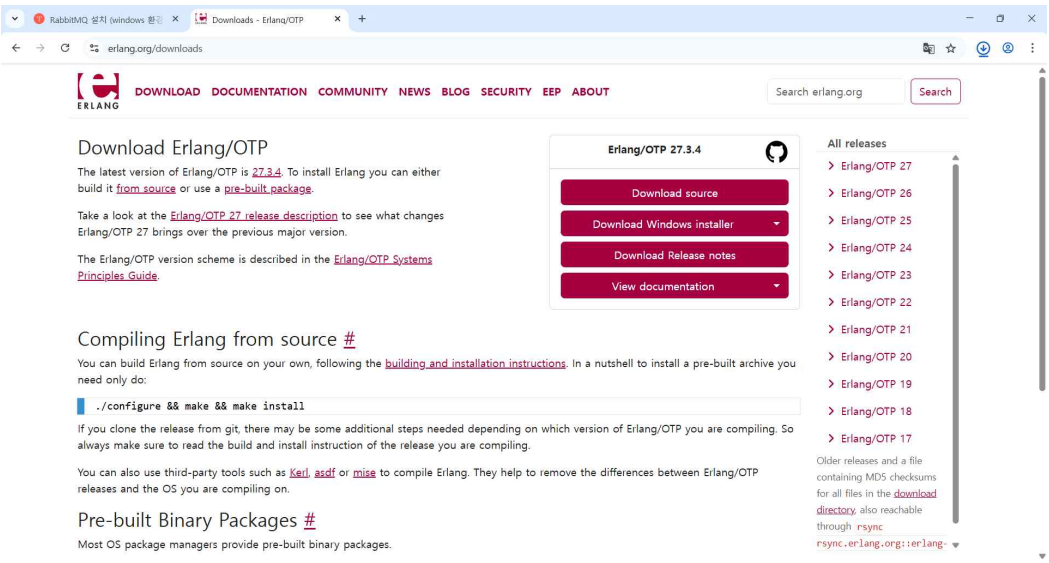

MySQL Workbench
DataEventTracking
Administration
Schemas
create_table
SCHEMAS
Filter objects
dataEventTrackings
Tables
data_event_tracking_request_info
event_occured_info
event_transmission_result_info
tracking_detector_proc_info
tracking_process_info
tracking_process_status_chang...
tracking_response_data
tracking_response_record
tracking_result_data_transmissi...
tracking_status_change_record
tracking_topic_info
Views
Stored Procedures
Functions
sys
Don't Limit
1 CREATE DATABASE IF NOT EXISTS `dataEventTrackings` /*!40100 DEFAULT CHARACTER SE
2 USE `dataEventTrackings`;
3 -- MySQL dump 10.13 Distrib 8.0.30, for Win64 (x86_64)
4 --
5 -- Host: localhost Database: dataEventTrackings
6 -- --
7 -- Server version 5.7.30
8
9 /*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
10 /*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
11 /*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
12 /*!50503 SET NAMES utf8 */;
13
100%
1:8
Action Output
Time
Action
Response
Duration / Fetch Time
67 14:22:11 /*!40101 SET charac... 0 row(s) affected, 1 warning(s): 1287 'utf8mb3' is dep... 0.00011 sec
68 14:22:11 /*!40103 SET TIME_... 0 row(s) affected 0.00026 sec
69 14:22:11 /*!40101 SET SQL_... 0 row(s) affected 0.00014 sec
70 14:22:11 /*!40014 SET FOREI... 0 row(s) affected 0.00013 sec
71 14:22:11 /*!40014 SET UNIQ... 0 row(s) affected 0.00012 sec
72 14:22:11 /*!40101 SET CHAR... 0 row(s) affected 0.00012 sec
73 14:22:11 /*!40101 SET CHAR... 0 row(s) affected 0.00010 sec
74 14:22:11 /*!40101 SET COLLA... 0 row(s) affected 0.00010 sec
75 14:22:11 /*!40111 SET SQL_N... 0 row(s) affected 0.000085 sec
Query Completed

5) 테이블 정보 확인



바. RabbitMQ 세팅

1) Erlang 설치 (https://erlang.org)

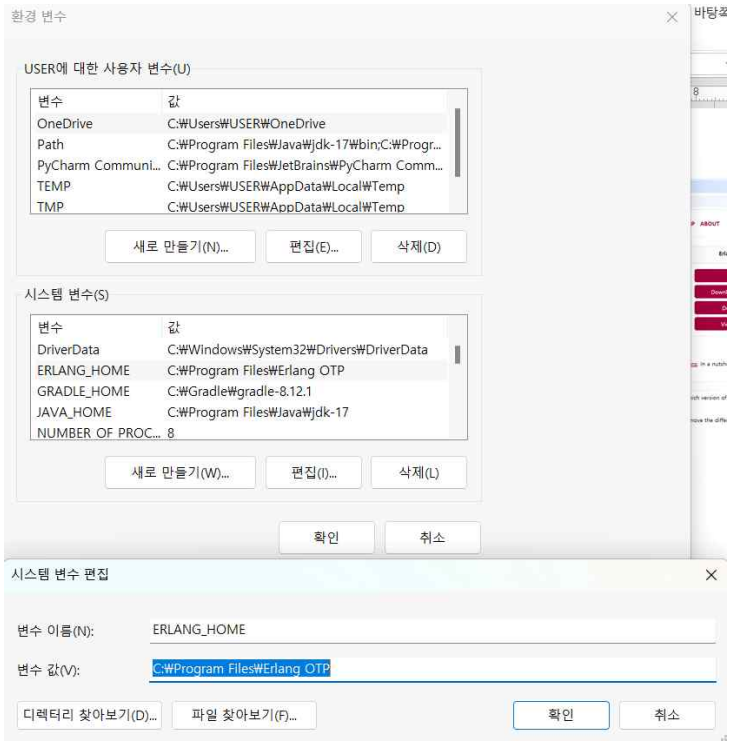


2) 환경 변수 등록

내 컴퓨터 - 속성 - 고급 시스템 설정 - 환경변수 - 시스템 변수 - 새로 만들기

변수 이름 : ERLANG_HOME

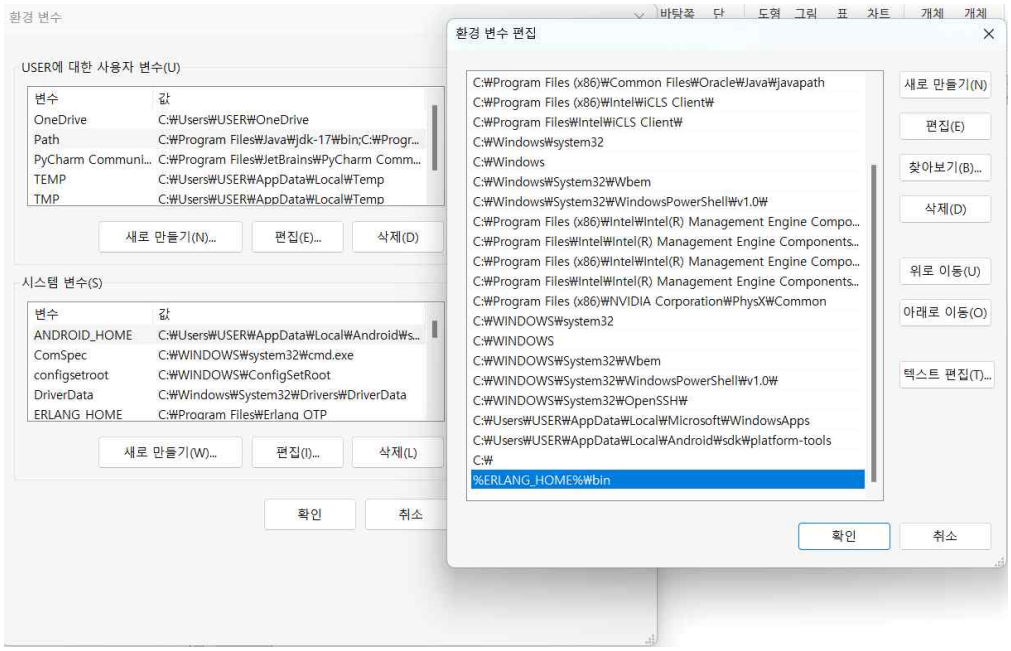
변수 값 : C:\Program Files\Erlang OTP



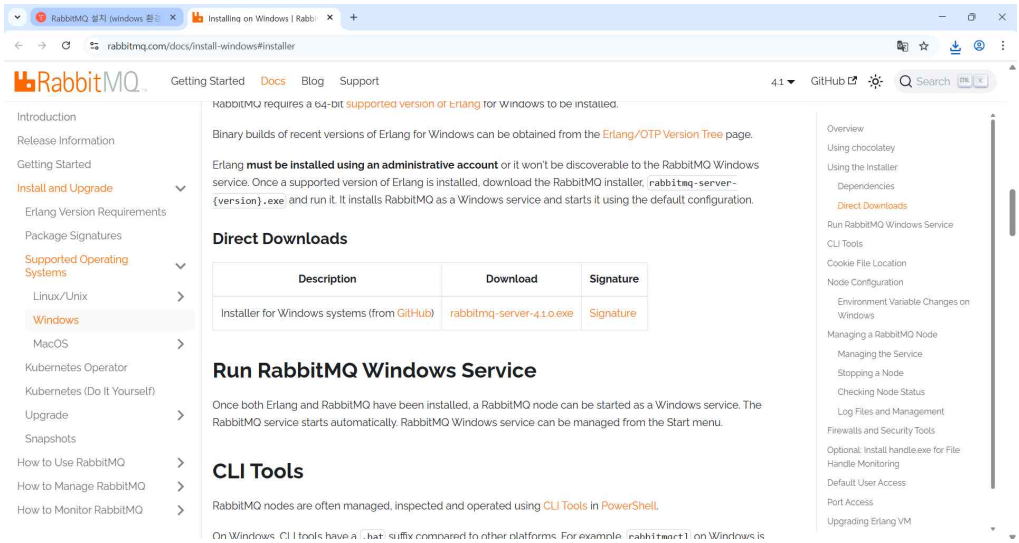
path 변수 경로 추가

사용자 변수 - path - 편집

path 변수 경로 추가 : %ERLANG_HOME%\bin

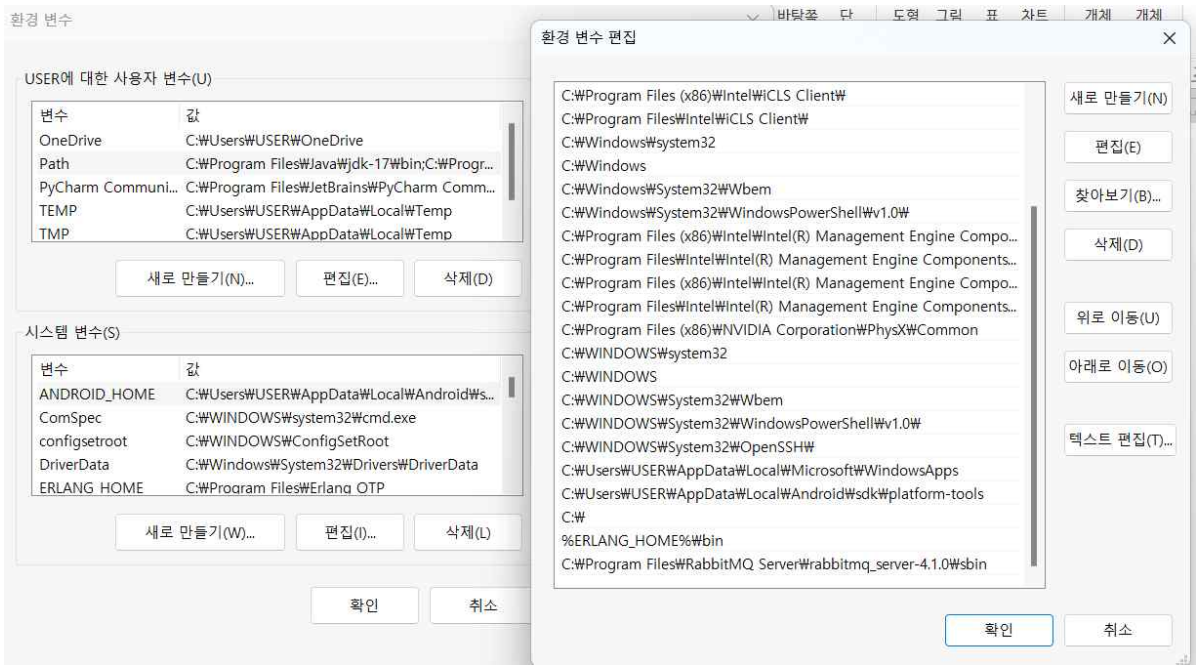


3) RabbitMQ 설치 (https://rabbitmq.com/install-windows.html#installer)

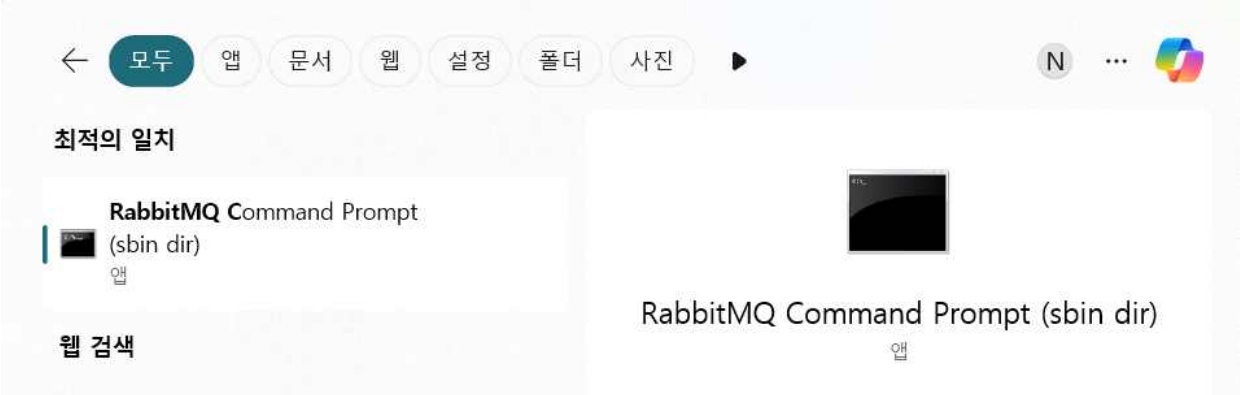


4) RabbitMQ 환경 변수 등록

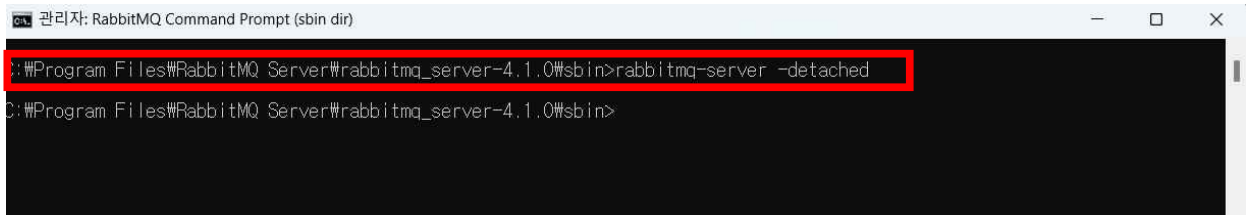
path 변수 경로 추가 : C:\Program Files\RabbitMQ Server\rabbitmq_server-4.1.0\sbin



5) RabbitMQ 백그라운드 서버 실행



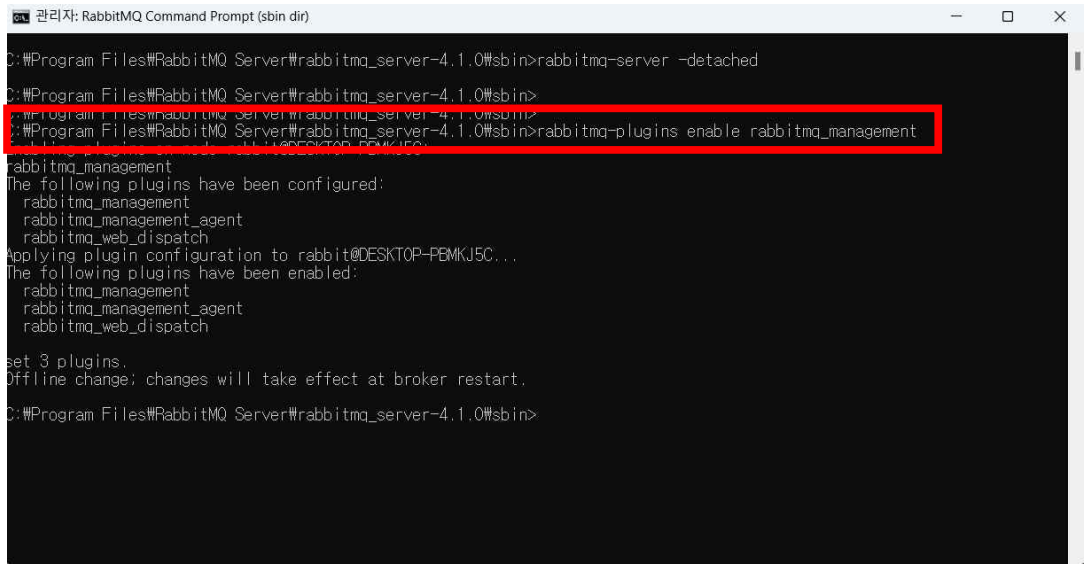
rabbitmq-server -detached 입력



6) Management Plugin 설치

windows powershell에 명령어 입력

rabbitmq-plugins enable rabbitmq_management



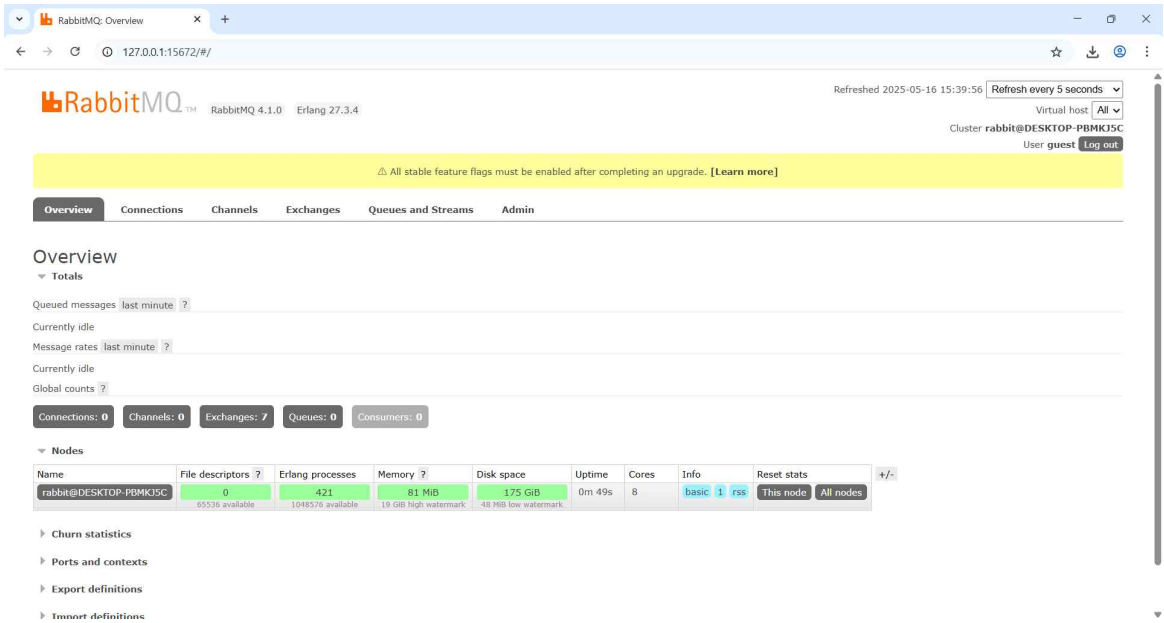
7) RabbitMQ 대시보드

http://127.0.0.1:15672/

초기 아이디 : guest

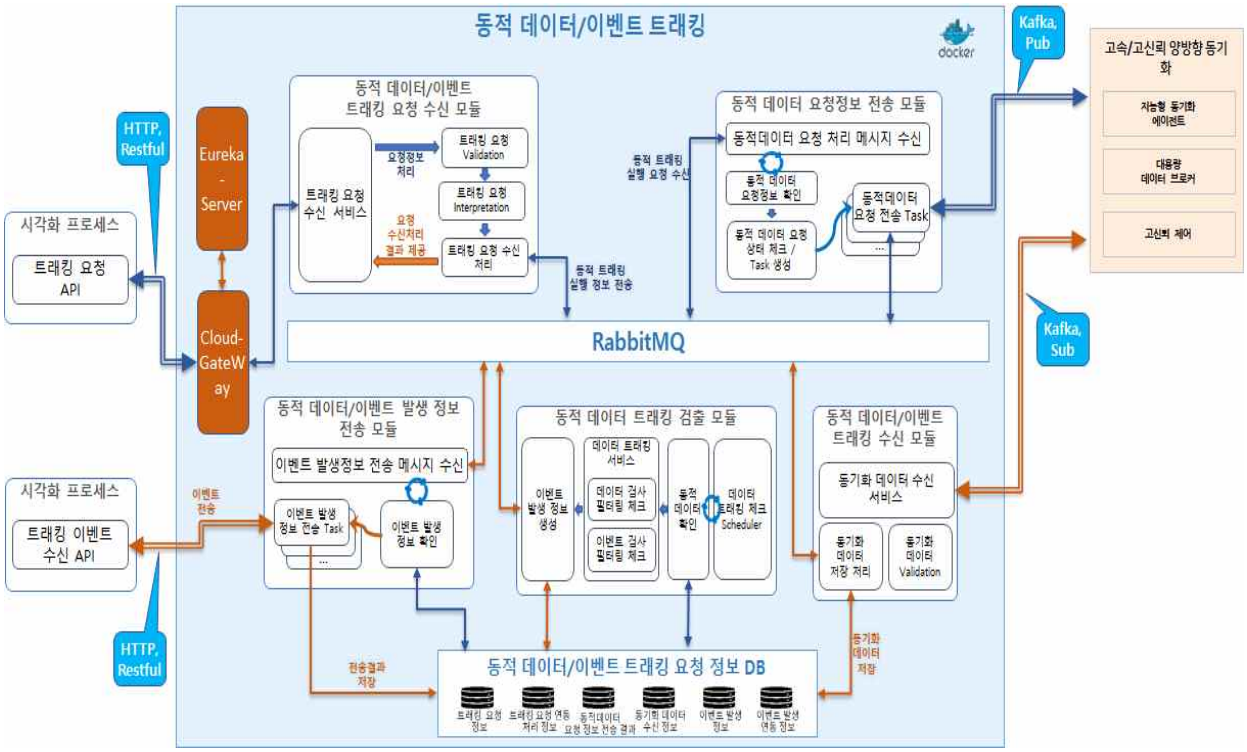
초기 비밀번호 : guest


RabbitMQ Management
127.0.0.1:15672
RabbitMQ
Username:
Password:
Login


RabbitMQ: Overview
127.0.0.1:15672/#/
RabbitMQ
RabbitMQ 4.1.0
Erlang 27.3.4
Refreshed 2025-05-16 15:39:56
Refresh every 5 seconds
Virtual host
All
Cluster rabbit@DESKTOP-PBMKJ5C
User guest
Log out
⚠ All stable feature flags must be enabled after completing an upgrade. [Learn more]
Overview
Connections
Channels
Exchanges
Queues and Streams
Admin
Overview
Totals
Queued messages last minute ?
Currently idle
Message rates last minute ?
Currently idle
Global counts ?
Connections: 0
Channels: 0
Exchanges: 7
Queues: 0
Consumers: 0
Nodes
Name
File descriptors ?
Erlang processes
Memory ?
Disk space
Uptime
Cores
Info
Reset stats
+/-
rabbit@DESKTOP-PBMKJ5C
0
65536 available
421
1048576 available
81 MiB
19 GiB high watermark
175 GiB
48 MiB low watermark
0m 49s
8
basic 1 rss
This node
All nodes
Churn statistics
Ports and contexts
Export definitions
Import definitions

3. 모듈 별 기능 구성 및 설정 정보

가. 동적 데이터/이벤트 트래킹 S/W구성 및 기능 구성



< Data/Event Tracking S/W 구성도 >

1) 업무 프로세스별 기능 모듈 구성

가) 업무 프로세스 구성

- Spring Boot Maven Multi Module 구조

나) 업무 모듈 구성

・업무 프로세스 모듈 : 동적 데이터/이벤트 트래킹 공통 모듈(Common)
Spring Cloud Eureka 모듈(DETRCKDiscoveryService)
Spring Cloud Api Gateway 모듈(DETRCKGatewayService)
동적 데이터/이벤트 트래킹 요청 수신 모듈(TrackCommandReceiver)
동적 데이터 요청정보 전송 모듈(TrackingCommandPublisher)
동적 데이터/이벤트 트래킹 수신 모듈(TrackingDataSubscriber)
동적 데이터/이벤트 발생 정보 전송 모듈(TrackingEventTrigger)
동적 데이터/이벤트 검출 모듈(TrackingCommandStatusChecker)

다) 내/외부 인터페이스

・내부 모듈 간 인터페이스 : AMQP(RabbitMQ) 사용

・외부 모듈 간 인터페이스 : 트래킹 요청(Http : Restful),
고속/고신뢰 양방향 동기화 엔진 : Kafka

2) 업무 프로세스 모듈 상세

가) 트래킹 업무 공통 모듈(Common)

· 모듈 정의

- 각 모듈에서 사용하는 공통 기능을 관리하는 모듈로서 공통모듈에서 사용하는 기능은 확장 가능한 기능들로 설정되어 있음.

· 모듈 기능

- DB Connection Pool(HikariCp), Mybatis 연동 Mapper, dto 및 Resource 설정
- 모듈에서 공통 사용가능 한 abstract class, interface 정의
- Sequence Task abstract Class 및 공통 정보처리 Service Interface 정의 및 기능 제공
- 공통 사용 Util, Exception 등 Object 정의 및 기능 제공
- 공통 Resource 정보 정의 및 설정

나) Spring Cloud Eureka 모듈(DETRCKDiscoveryService)

· 모듈 정의

- 클라우드 배포(Docker) 시 Eureka 서버에 등록된 정보를 통해 동적으로 찾을 수 있으며 하드 코딩된 주소를 없애고 유연한 구조를 제공 함

· 모듈 기능

- Eureka Server 기능의 모듈로서 Eureka Client의 Instance를 관리
- Spring Cloud Api Gateway와 연동 및 Eureka Client 요청 정보 전달 기능

· application.properties 설정 정보

```
pring.application.name=eureka-server
server.port=8761
eureka.instance.hostname={server ip}
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
eureka.client.service-url.defaultZone=http://${eureka.instance.hostname}:${server.port}/eureka/

eureka.server.waitTimeInMsWhenSyncEmpty=5
eureka.server.enable-self-preservation=true

eureka.server.response-cache-update-interval-ms=3000

eureka.server.eviction-interval-timer-in-ms=10000

logging.level.com.netflix.eureka=OFF
logging.level.com.netflix.discovery=OFF

spring.security.user.name={eurekaId}
spring.security.user.password={eurekaPw}
```

server.port	Eureka server 서비스 포트(기본 포트 : 8761)
eureka.instance.hostname	Eureka server 구동 실행 IP(local 구동 시 localhost, 외부 서버 구동 시 서버 IP)
spring.security.user.name	Eureka Client 모듈이 Server에 접속 시 인증정보 (ID)
spring.security.user.password	Eureka Client 모듈이 Server에 접속 시 인증정보 (Passwd)

(a) Spring Cloud Api Gateway 모듈(DETRCKGatewayService)

· 모듈 정의

- 클라우드 기반 배포 시 Client와 Back-End 사아에 진입점(End Point) 및 프록시 역할을 하는 서버

· 모듈 기능

- 인증처리 및 라우팅 등 공통기능 처리, 확장을 위한 비동기적인 API 라우팅 및 필터링 기능을 제공합니다.

· application.properties 설정 정보

```
spring.application.name=eureka-gateway-service

spring.security.user.name={eurekaId}
spring.security.user.password={eurekaPw}

eureka.instance.hostname={server ip}
eureka.instance.preferIpAddress=true
eureka.client.register-with-eureka=true
eureka.client.fetch-registry=true
eureka.client.service-url.defaultZone=http://${spring.security.user.name}:${spring.security.user.password}@{server ip}:8761/eureka/

eureka.client.registry-fetch-interval-seconds=3
eureka.client.disable-delta=true

eureka.instance.lease-renewal-interval-in-seconds=10
eureka.instance.lease-expiration-duration-in-seconds=20

eureka.instance.instance-id=${spring.cloud.client.ip-address}:${spring.application.name}:${spring.application.instance_id:${server.port}}

#application.properties
management.endpoints.web.exposure.include=health,info
management.endpoint.health.show-details=always


server.port=8000

app.version: 2.0
```

```
#RabbitMQ Config
spring.rabbitmq.host:{server ip}
spring.rabbitmq.port:5672
spring.rabbitmq.username:guest
spring.rabbitmq.password:guest

rabbitmq.exchange.name:federate.tracking
rabbitmq.queue.name:topic.publish.rec
rabbitmq.routing.key:topic.publish.#

#msa routes
spring.cloud.loadbalancer.ribbon.enabled=true

spring.cloud.gateway.routes[0].id=TrackingCommandReceiver
spring.cloud.gateway.routes[0].uri=lb://TrackingCommandReceiver
spring.cloud.gateway.routes[0].predicates[0].name=Path
spring.cloud.gateway.routes[0].predicates[0].args.pattern=/fdt/detrck/rcv/**

spring.kafka.bootstrap-servers:{server ip}:9092
spring.kafka.cloud.bus.enabled: true

# Logging
logging.level.gateway.filters.JwtAuthFilter=info
logging.file.name=/var/log/${spring.application.name}.log
logging.file.max-size=100MB
logging.file.max-history=5


# Routing
spring.cloud.gateway.loadbalancer.use404=true
```

server.port	Api Gateway 서비스 포트(기본 포트 : 8000)
spring.security.user.name	Eureka Server 접속 시 사용할 인증 ID (Eureka Server 설정 정보와 동일한 정보 사용)
spring.security.user.password	Eureka Server 접속 시 사용할 인증 Passwd (Eureka Server 설정 정보와 동일한 정보 사용)

eureka.instance.hostname	Eureka server 구동 실행 IP(local 구동 시 localhost, 외부 서버 구동 시 서버 IP)
eureka.client.service-url.defaultZone	Eureka server 구동 실행 IP, Port(local 구동 시 localhost, 외부 서버 구동 시 서버 IP)
RabbitMQ Config 정보 셋팅	RabbitMQ 구동 실행 중인 IP, Port, 접속 정보(local 구동 시 localhost, 외부 서버 구동 시 서버 IP)
spring.kafka.bootstrap-servers	고속/고신뢰 동기화 엔진(Kafka) 실행 IP 및 서비스 포트

(b) 동적 데이터/이벤트 트래킹 요청 수신 모듈(TrackCommandReceiver)

・모듈 정의

- 3세부에서 요청하는 동적 데이터/이벤트 트래킹 요청정보를 수신하며 수신되는 요청정보를 파싱 및 유효성 체크 및 처리 Task Indexing 작업 진행

・ 모듈 기능

- TrackingCommandFactory , TrackingCommandValidator, 유효성 처리 Filter 기능
- 동적 트래킹 요청 수신처리 및 Business Logic 기능 처리를 위한 controller, service, mapper, vo 기능

・ application.properties 설정 정보

```
spring.application.name=TrackingCommandReceiver

spring.security.user.name={eurekaId}
spring.security.user.password={eurekaPw}

eureka.instance.hostname={server ip}
eureka.instance.preferIpAddress=true
eureka.client.register-with-eureka=true
eureka.client.fetch-registry=true
eureka.client.service-url.defaultZone=http://${spring.security.user.name}:${spring.security.user.password}@{server ip}:8761/eureka/

eureka.client.registry-fetch-interval-seconds=3
eureka.client.disable-delta=true

eureka.instance.lease-renewal-interval-in-seconds=10
eureka.instance.lease-expiration-duration-in-seconds=20

eureka.instance.instance-id=${spring.cloud.client.ip-address}:${spring.application.name}:${spring.application.instance_id:${server.port}}

#application.properties
management.endpoints.web.exposure.include=refresh,health,beans,busenv,busrefresh,info,metrics,prometheus,customEndpoint
management.endpoint.health.show-details=always

#RabbitMQ Config
spring.rabbitmq.host:{server ip}
spring.rabbitmq.port:5672
spring.rabbitmq.username:guest
spring.rabbitmq.password:guest

rabbitmq.exchange.name:federate.tracking
rabbitmq.queue.name:tracking.receiver.rec
rabbitmq.routing.key:tracking.receiver.rec.#

app.version: 2.0


CTRL_CMD_REQ_RECORDING_TYPE=TRUE

server.port=22419

spring.datasource.hikari.jdbc-url=jdbc:mysql://{server ip}:{server
port}/federatedTwins?autoCommit=false&useUnicode=true&characterEncoding=utf8&useSSL=false&serverTimezone=UTC&autoReconnect=true

spring.datasource.hikari.username={userId}
spring.datasource.hikari.password={userPw}
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.minimumidle=10
spring.datasource.hikari.connection-test-query=SELECT 1
```

server.port	TrackCommandReceiver 모듈 서비스 포트(기본 포트 : 22419)

eureka.instance.hostname	Eureka server 구동 실행 IP(local 구동 시 localhost, 외부 서버 구동 시 서버 IP)
spring.security.user.name	Eureka Client 모듈이 Server에 접속 시 인증정보 (ID)
spring.security.user.password	Eureka Client 모듈이 Server에 접속 시 인증정보 (Passwd)
eureka.client.service-url.defaultZone	Eureka server 구동 실행 IP, Port(local 구동 시 localhost, 외부 서버 구동 시 서버 IP)
RabbitMQ Config 정보 셋팅	RabbitMQ 구동 실행 중인 IP, Port, 접속 정보(local 구동 시 localhost, 외부 서버 구동 시 서버 IP)
spring.datasource.hikari	DB접속 및 사용을 위하여 Hicari CP를 사용 환경 설정에서 설치/구성한 MariaDB 접속 정보 셋팅

(c) 동적 데이터 요청정보 전송 모듈(TrackingCommandPublisher)

・모듈 정의

- 수신된 요청정보를 동적 트래킹 단위 시퀀스로 분할하여 처리 고속/고신뢰 양방향 동기화 엔진에 동적 데이터 요청 / 트래킹 시퀀스 요청(pub/sub) 처리

・ 모듈 기능

- 동적 트래킹 요청처리를 위한 Scheduler(요청 분배, Task 실행, Task 전송) 기능
- 트래킹 Task 요청처리 및 Business Logic 기능 처리를 위한 service, mapper, vo 기능
- 동적 트래킹 단위 시퀀스 요청처리를 위한 Task process 객체 기능 및 고속/고신뢰 양방향 동기화 엔진에 트래킹 요청정보를 요청하는 pub/sub Task 기능

・ application.properties 설정 정보

```
spring.application.name=TrackingCommandPublisher

#RabbitMQ Config
spring.rabbitmq.host:{server ip}
spring.rabbitmq.port:5672
spring.rabbitmq.username:guest
spring.rabbitmq.password:guest

rabbitmq.exchange.name:federate.tracking
rabbitmq.queue.name:tracking.publish.pub
rabbitmq.routing.key:tracking.publish.pub.#

spring.kafka.bootstrap-servers:{server ip}:9092
spring.kafka.cloud.bus.enabled: true

server.port=22429

spring.datasource.hikari.jdbc-url=jdbc:mysql://{server ip}:{server
port}/federatedTwins?autoCommit=false&useUnicode=true&characterEncoding=utf8&useSSL=false&serverTimezone=UTC&autoReconnect=true

spring.datasource.hikari.username={userId}
spring.datasource.hikari.password={userPw}
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.connection-test-query=SELECT 1


logging.config=classpath:logback/logback.xml


mybatis.mapper-locations=classpath:publisher/mybatis/mapper/**/*.xml
```

server.port	TrackingCommandPublisher 모듈 서비스 포트(기본 포트 : 22429)
RabbitMQ Config 정보 셋팅	RabbitMQ 구동 실행 중인 IP, Port, 접속 정보(local 구동 시 localhost, 외부 서버 구동 시 서버 IP)

spring.kafka.bootstrap-servers	고속/고신뢰 동기화 엔진(Kafka) 실행 IP 및 서비스 포트
spring.datasource.hikari	DB접속 및 사용을 위하여 Hicari CP를 사용 환경 설정에서 설치/구성한 MariaDB 접속 정보 셋팅

(d) 동적 데이터/이벤트 트래킹 수신 모듈(TrackingDataSubscriber)

· 모듈 정의

- 고속/고신뢰 양방향 동기화 엔진에 요청한 동적데이터를 수신하는 모듈로서 수신되는 정보를 반영 처리 기능 제공

· 모듈 기능

- 동적데이터 처리 결과 수신처리 및 Business Logic 기능 처리를 위한 controller, service, mapper, vo 기능 구현
- 데이터 유효성 처리를 위한 Filter 기능

· application.properties 설정 정보

```
spring.application.name=TrackingDataSubscriber

#RabbitMQ Config
spring.rabbitmq.host:{server ip}
spring.rabbitmq.port:5672
spring.rabbitmq.username:guest
spring.rabbitmq.password:guest

rabbitmq.exchange.name:federate.tracking
rabbitmq.queue.name:tracking.subscriber.sub
rabbitmq.routing.key:tracking.Subscriber.sub.#


spring.kafka.bootstrap-servers:{server ip}:9092
spring.kafka.cloud.bus.enabled: true

server.port=22439

## Linux Centos7 parameter
script.engine.name:nashorn

spring.datasource.hikari.jdbc-url=jdbc:mysql://{server ip}:{server
port}/federatedTwins?autoCommit=false&useUnicode=true&characterEncoding=utf8&useSSL=false&serverTimezone=UTC&autoReconnect=true

spring.datasource.hikari.username={userId}
spring.datasource.hikari.password={userPw}
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.connection-test-query=SELECT 1

logging.config=classpath:logback/logback.xml
mybatis.mapper-locations=classpath:subscriber/mybatis/mapper/**/*.xml
```

server.port	TrackingDataSubscriber 모듈 서비스 포트(기본 포트 : 22439)
RabbitMQ Config 정보 셋팅	RabbitMQ 구동 실행 중인 IP, Port, 접속 정보(local 구동 시 localhost, 외부 서버 구동 시 서버 IP)
spring.kafka.bootstrap-servers	고속/고신뢰 동기화 엔진(Kafka) 실행 IP 및 서비스 포트
spring.datasource.hikari	DB접속 및 사용을 위하여 Hicari CP를 사용 환경 설정에서 설치/구성한 MariaDB 접속 정보 셋팅

(e) 동적 데이터/이벤트 검출 모듈(TrackingCommandStatusChecker)

· 모듈 정의

- 수신된 동적데이터를 스케줄링 검출모듈을 활용하여 데이터/이벤트 정보를 검출 진행 및 검출 시 Filtering 체크 확인을 통하여 이벤트 정보를 발생하는 기능 제공

· 모듈 기능

- 동적데이터 검출을 위한 Scheduler 기능
- 데이터/이벤트 트래킹 정보 확인 및 이벤트 발생처리 및 Business Logic 기능 처리를 위한 service, mapper, vo 기능
- 트래킹 검출처리 위한 Task process 객체 기능

· application.properties 설정 정보

```
spring.application.name=TrackingCommandStatusChecker

#RabbitMQ Config
spring.rabbitmq.host:{server ip}
spring.rabbitmq.port:5672
spring.rabbitmq.username:guest
spring.rabbitmq.password:guest

rabbitmq.exchange.name:federate.tracking
rabbitmq.queue.name:topic.publish.che
rabbitmq.routing.key:topic.publish.che.#

server.port=22459

spring.datasource.hikari.jdbc-url=jdbc:mysql://{server ip}:{server
port}/federatedTwins?autoCommit=false&useUnicode=true&characterEncoding=utf8&useSSL=false&serverTimezone=UTC&autoReconnect=true

spring.datasource.hikari.username={userId}
spring.datasource.hikari.password={userPw}
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.connection-test-query=SELECT 1

logging.config=classpath:logback/logback.xml

mybatis.mapper-locations=
```

server.port	TrackingCommandStatusChecker 모듈 서비스 포트(기본 포트:22439)
RabbitMQ Config 정보 셋팅	RabbitMQ 구동 실행 중인 IP, Port, 접속 정보(local 구동 시 localhost, 외부 서버 구동 시 서버 IP)
spring.datasource.hikari	DB접속 및 사용을 위하여 Hicari CP를 사용 환경 설정에서 설치/구성한 MariaDB 접속 정보 셋팅

(f) 동적 데이터/이벤트 발생 정보 전송 모듈(TrackingEventTrigger)

· 모듈 정의

- 검출 모듈을 통하여 데이터/이벤트 트래킹 정보가 발생 시 해당하는 이벤트를 수신을 요청한 CallBackURL 정보를 활용하여 이벤트 발생 정보전송

· 모듈 기능

- 발생된 이벤트의 전송처리를 위한 Scheduler(전송 분배, 이벤트 전송 실행Task) 기능
- 이벤트 전송Task 처리 및 Business Logic 기능 처리를 위한 service, mapper, vo 기능
- 이벤트 전송처리를 위한 Event TriggerTask 기능(HttpRestful)

· application.properties 설정 정보

```
spring.application.name=TrackingEventTrigger

#RabbitMQ Config
spring.rabbitmq.host:{server ip}
spring.rabbitmq.port:5672
spring.rabbitmq.username:guest
spring.rabbitmq.password:guest

rabbitmq.exchange.name:federate.tracking
rabbitmq.queue.name:tracking.trigger.trn
rabbitmq.routing.key:tracking.trigger.trn.#

server.port=22449

spring.datasource.hikari.jdbc-url=jdbc:mysql://{server ip}:{server
port}/federatedTwins?autoCommit=false&useUnicode=true&characterEncoding=utf8&useSSL=false&serverTimezone=UTC&autoReconnect=true

spring.datasource.hikari.username={userId}
spring.datasource.hikari.password={userPw}
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.connection-test-query=SELECT 1

logging.config=classpath:logback/logback.xml
mybatis.mapper-locations=classpath:trigger/mybatis/mapper/**/*.xml
```

server.port	TrackingEventTrigger 모듈 서비스 포트 (기본 포트 : 22449)
RabbitMQ Config 정보 셋팅	RabbitMQ 구동 실행 중인 IP, Port, 접속 정보(local 구동 시 localhost, 외부 서버 구동 시 서버 IP)
spring.datasource.hikari	DB접속 및 사용을 위하여 Hicari CP를 사용 환경 설정에서 설치/구성한 MariaDB 접속 정보 셋팅

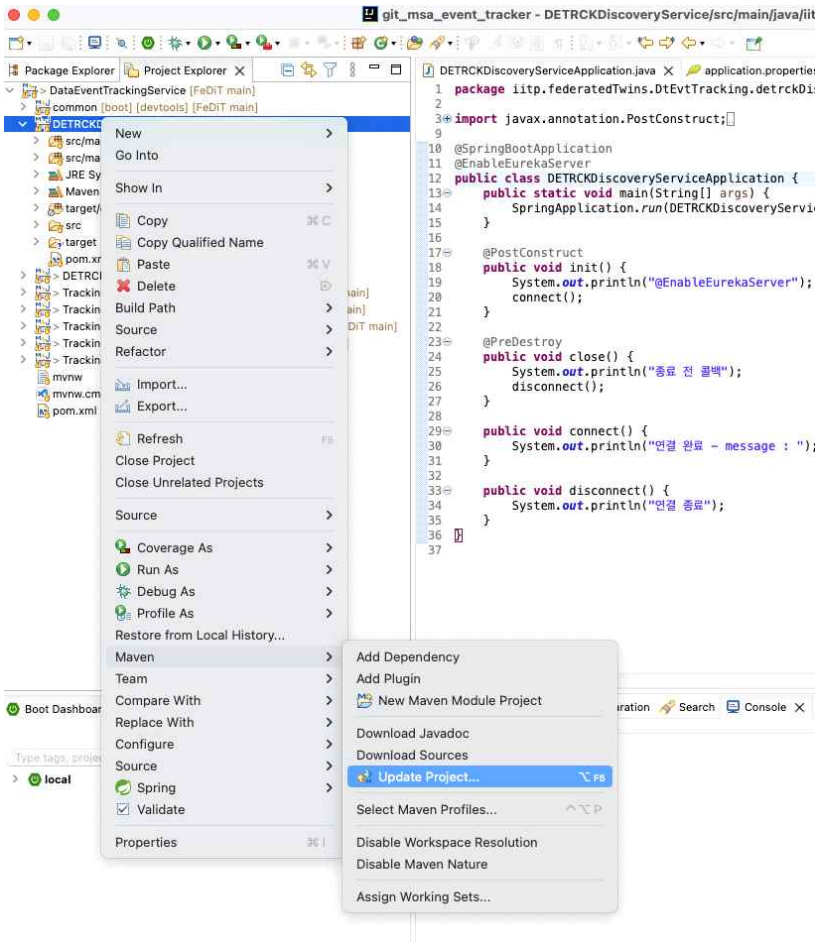
4. 구동/실행

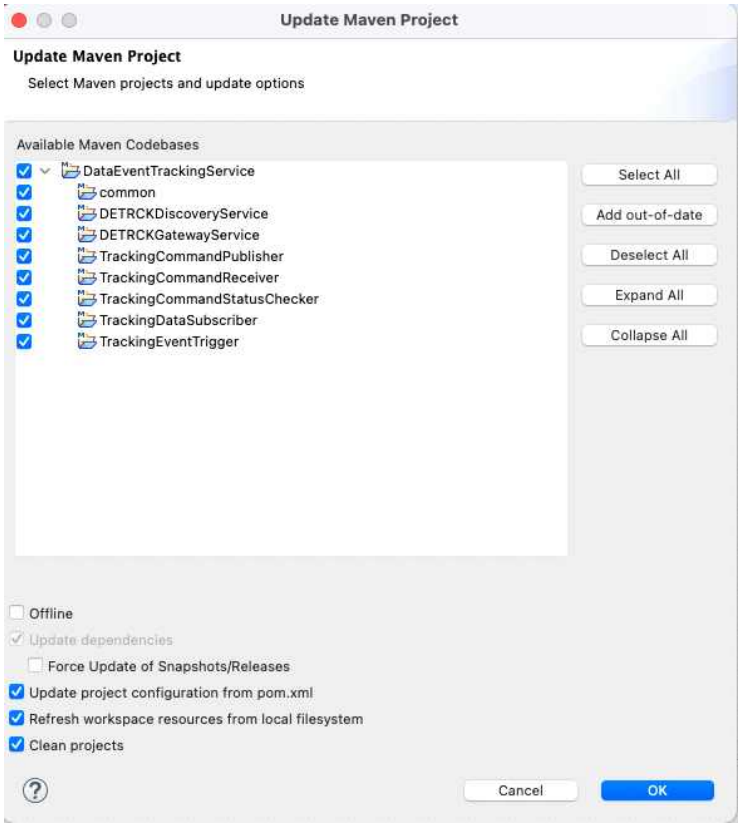
가. Data/Event Tracking Local 구동/실행

1) 구동 시 확인 사항

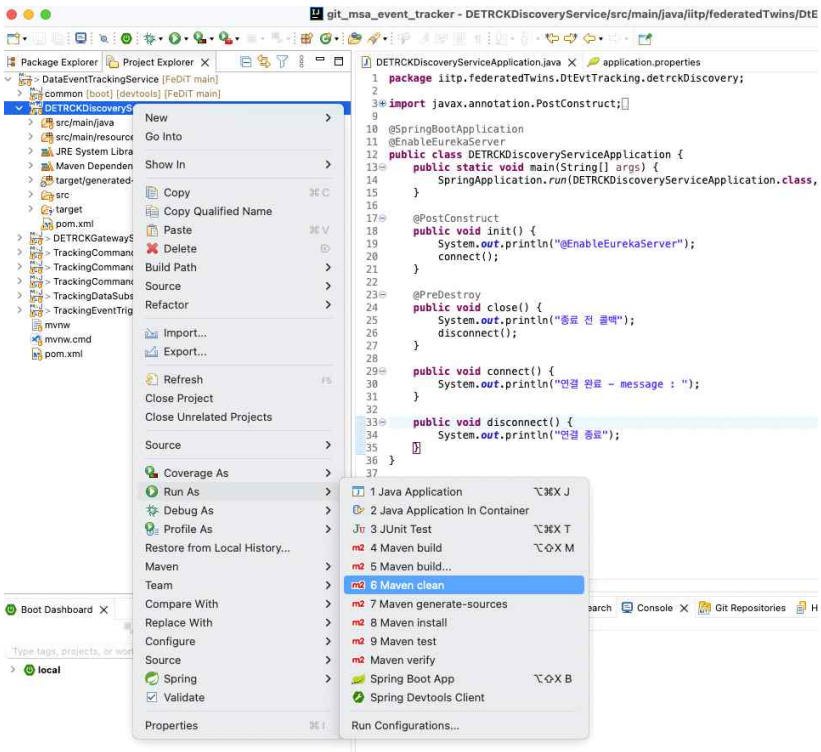
- 개발환경 설정정보를 기반으로 프로젝트 환경설정 정보 셋팅(application.properties)을 마무리하게 되면 아래의 작업을 진행하여 Local에서 트래킹 모듈을 실행하여 테스트할 수 있습니다.
- 테스트를 위해서는 고속/고신뢰 양방향 동기화 엔진과 연계가 필요하기 때문에 선 엔진의 서비스 정보를 확인 및 셋팅해야 합니다.
- Data/Event Tracking 수신용 CallBack 기능을 반드시 구현해야 하며 이 수신 정보(CallBack URL)는 Command 실행 시 필수 요소로 조합되며, Command가 정상적으로 수행되기 위해 반드시 필요합니다.

2) 모듈 프로젝트 우클릭 - Maven - Update Project

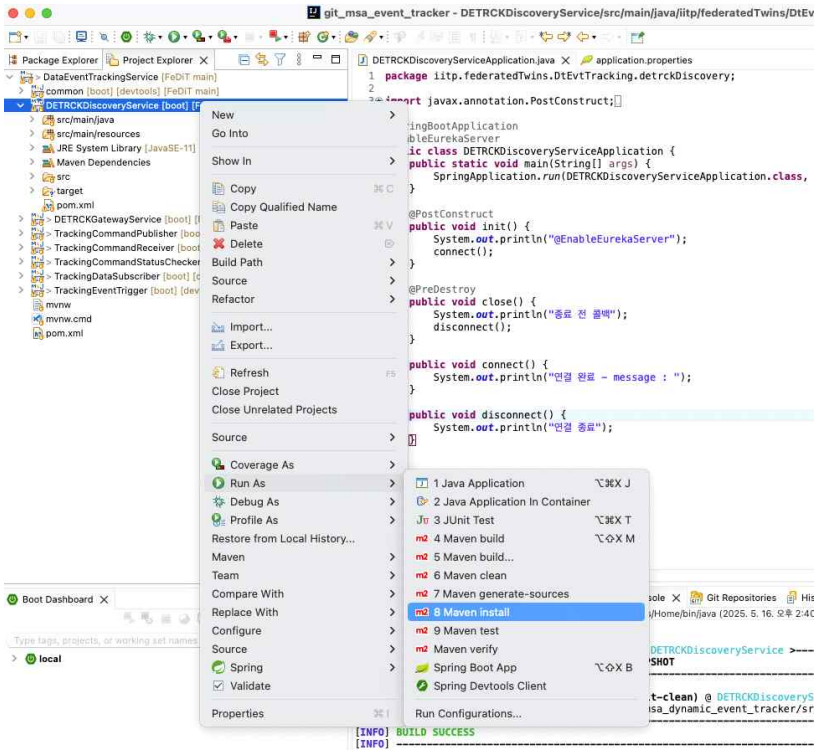




3) 모듈 프로젝트 우클릭 - Run As - Maven Clean



4) 모듈 프로젝트 우클릭 - Run As - Maven Install



5) Boot Dashboard - local - ResultDataConsumer 모듈 우클릭 - start



6) 실행 순서

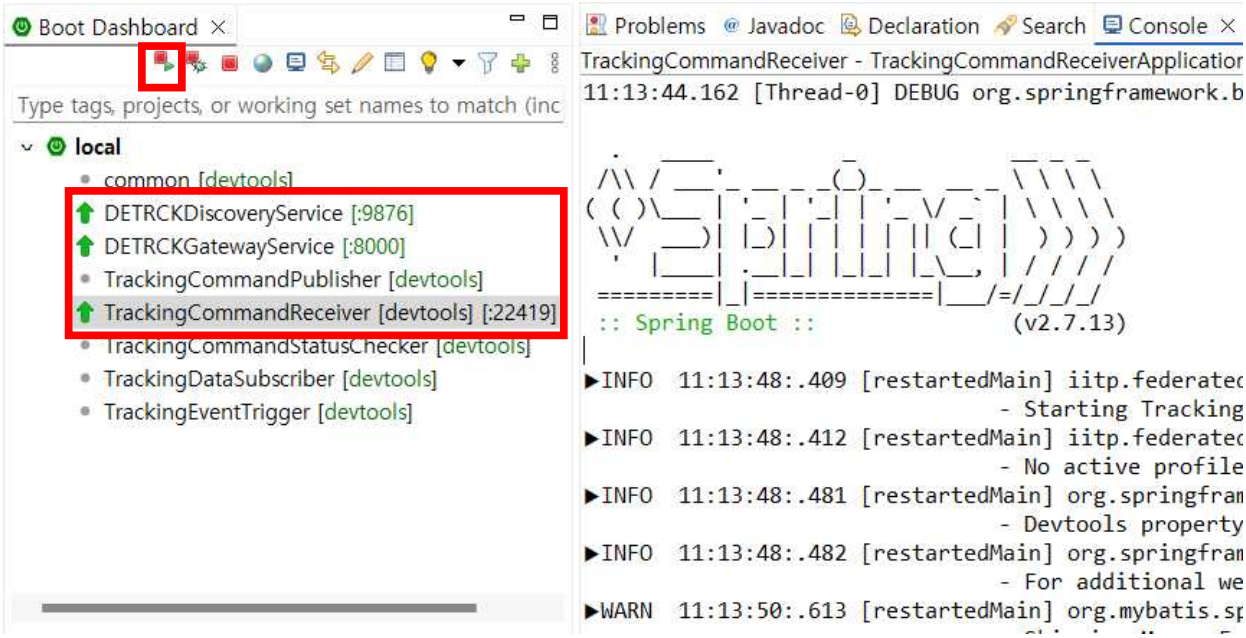
- 모듈 별 실행순서

DETRCKDiscoveryService -> DETRCKGatewayService -> TrackCommandReceiver -> TrackingCommandPublisher -> TrackingDataSubscriber -> TrackingEventTrigger -> TrackingCommandStatusChecker

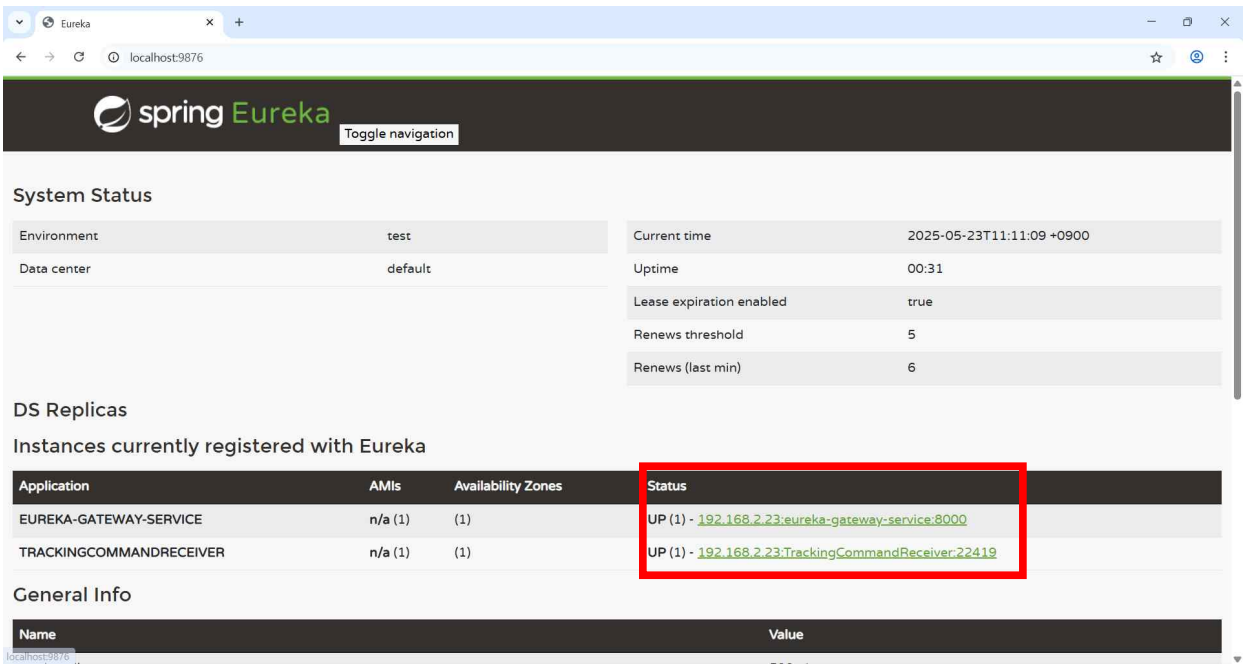
- Eureka Server, Client 연관 모듈 실행

(DETRCKDiscoveryService -> DETRCKGatewayService -> TrackCommandReceiver)

(1) 실행 화면



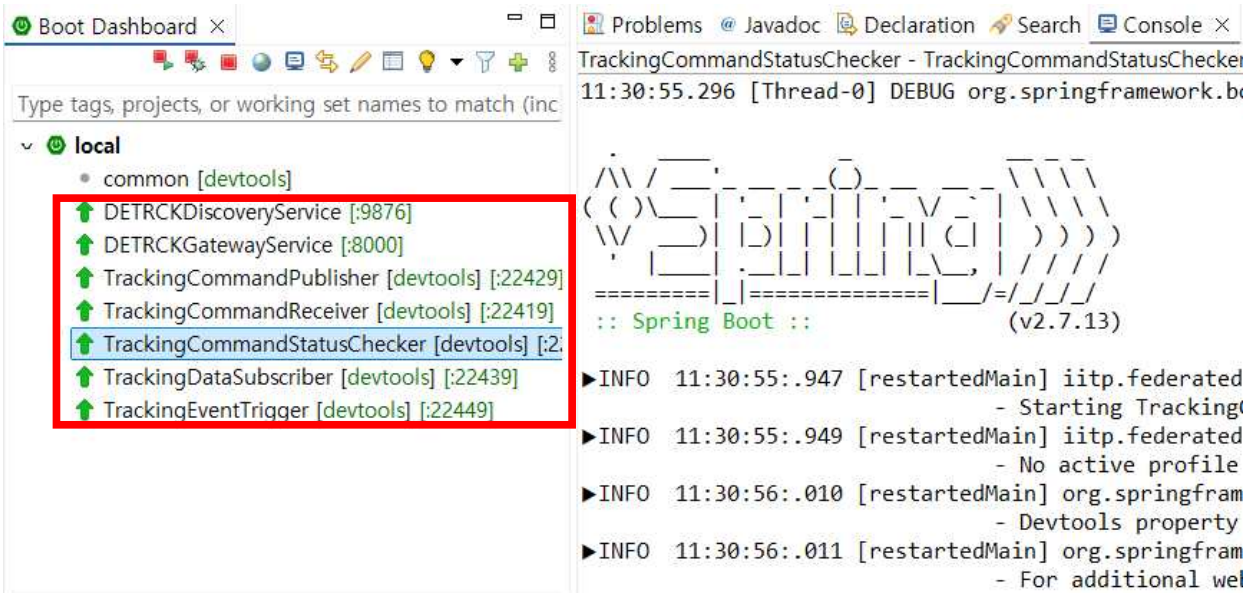
- Eureka 접속 화면



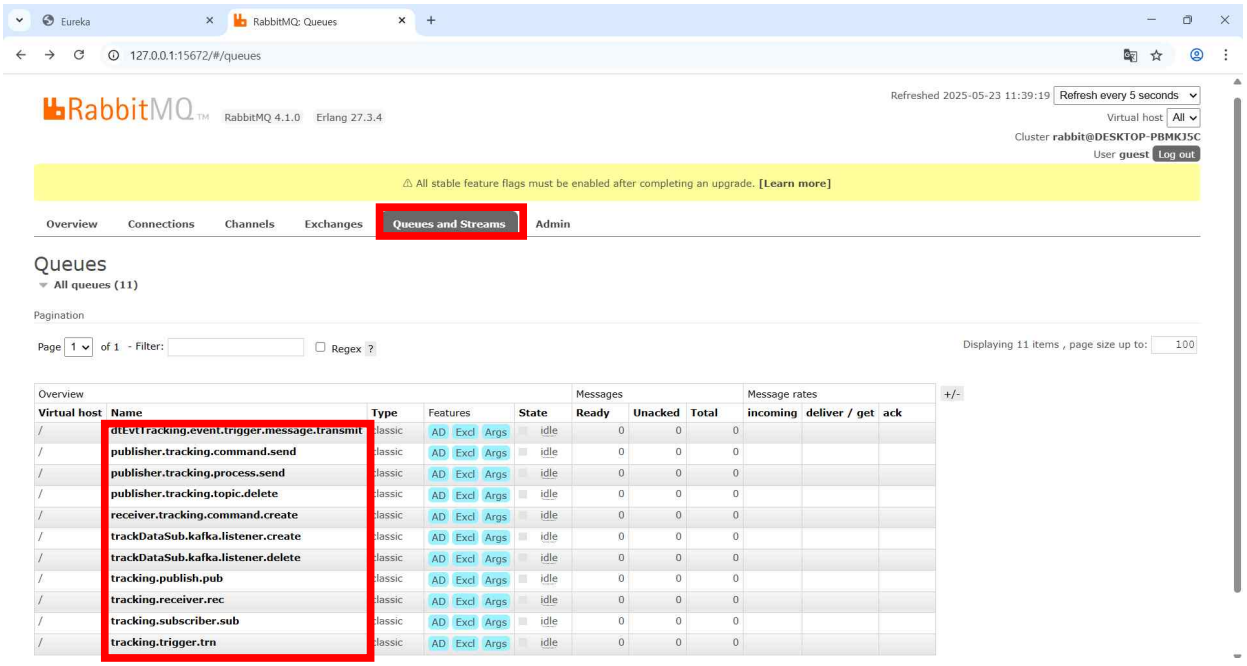
- Tracking 처리 프로세스 모듈 실행

(TrackingCommandPublisher -> TrackingDataSubscriber -> TrackingEventTrigger -> TrackingCommandStatusChecker)

(2) 실행 화면



- RabbitMQ 실행화면



7) Local 테스트

- Local 테스트 툴(PostMan)을 통하여 요청정보 및 트래킹 데이터 수신의 테스트를 진행할 수 있다.

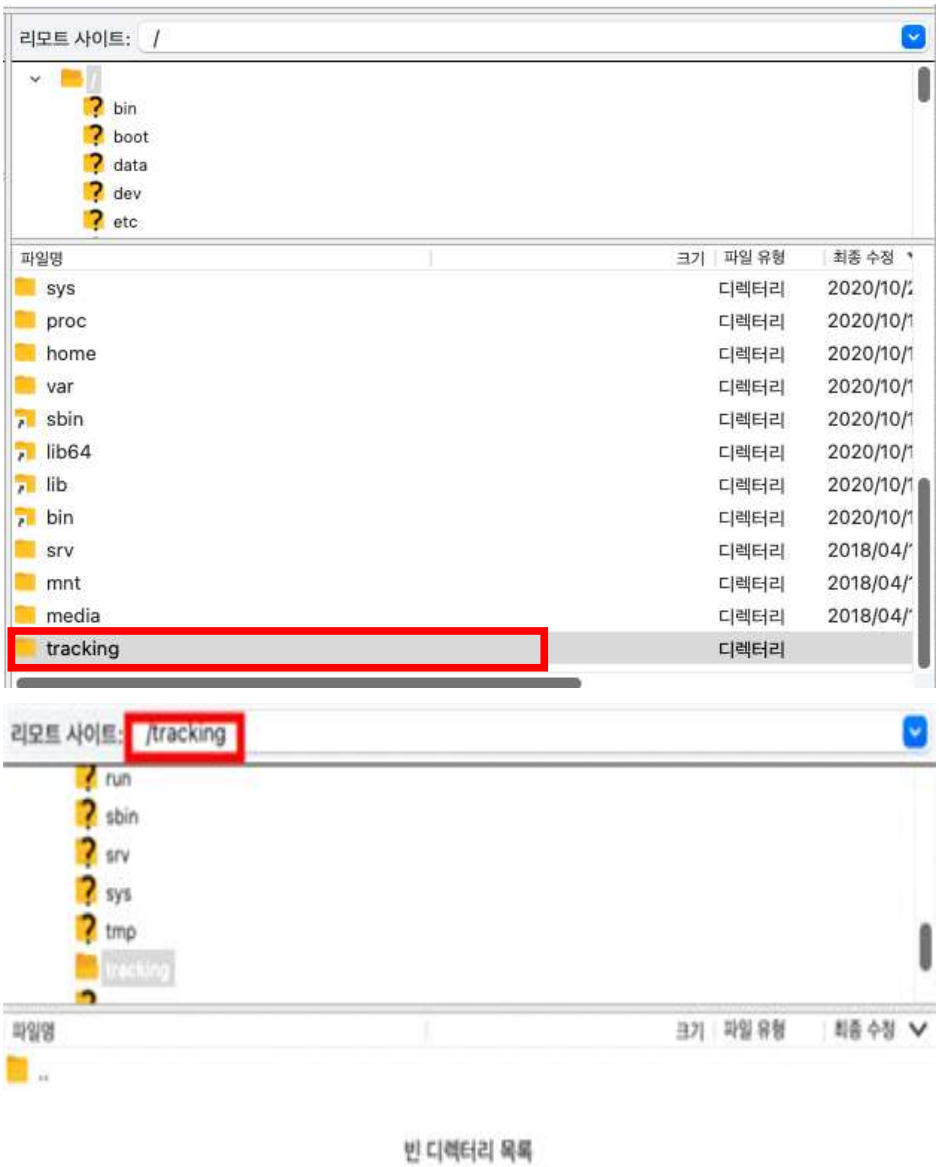
5. 동적 데이터/이벤트 트래킹 배포

가. 서버 배포 구성

1) Git에서 initResource/DataEventTrackingService 폴더 다운로드

2) FTP 서버 접속 (e.g. FileZilla)

3) 서버 하위에 하기와 같이 폴더 생성



빈 디렉터리 목록

4) Git에서 다운받은 DataEventTrackingService 파일을 /tracking 폴더 하위에 업로드

가) DataEventTrackingService 은 배포를 위한 설정파일, 실행파일 및 설정정보파일 등을 포함하는 폴더 구조의 파일

db	2025-05-16 오후 5:33	파일 폴더	
.env	2025-05-29 오후 5:22	AnySign4PC 암호...	1KB
.env.prod	2025-05-26 오전 8:55	PROD 파일	1KB
DETRCKDiscoveryService-0.0.1-SNAPSH...	2025-05-16 오후 5:33	Executable Jar File	55,667KB
DETRCKGatewayService-0.0.1-SNAPSHO...	2025-05-16 오후 5:33	Executable Jar File	48,853KB
docker-compose.yml	2025-05-20 오후 2:45	YML 파일	3KB
Dockerfile	2025-05-16 오후 5:33	파일	1KB
extStartUp.sh	2025-05-16 오후 5:27	Shell Script	2KB
TrackingCommandPublisher-0.0.1-SNAP...	2025-05-16 오후 5:29	Executable Jar File	44,364KB
TrackingCommandReceiver-0.0.1-SNAPS...	2025-05-16 오후 5:28	Executable Jar File	69,828KB
TrackingCommandStatusChecker-0.0.1-...	2025-05-16 오후 5:31	Executable Jar File	44,348KB
TrackingDataSubscriber-0.0.1-SNAPSHO...	2025-05-16 오후 5:31	Executable Jar File	81,216KB
TrackingEventTrigger-0.0.1-SNAPSHOT.jar	2025-05-16 오후 5:27	Executable Jar File	29,543KB

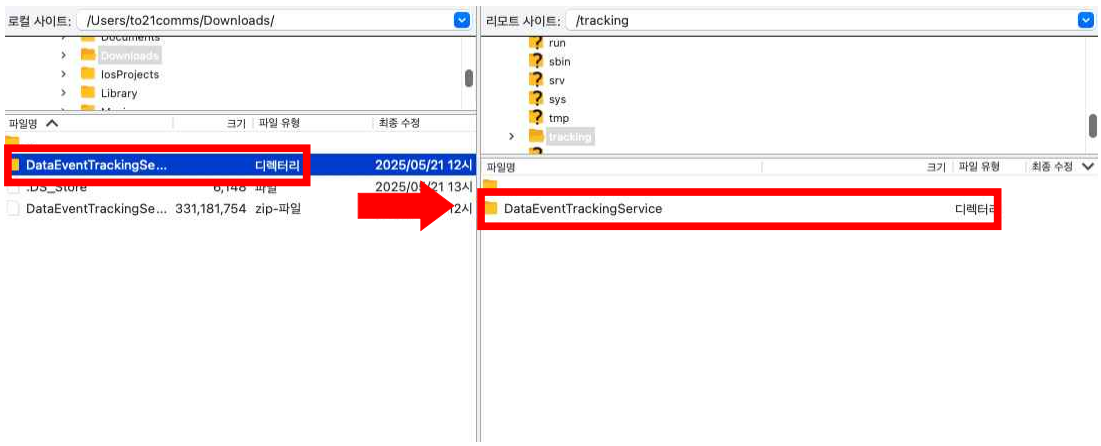
(1) 파일 정보

- 총 12종의 파일 및 1개 폴더로 구성 됨
- 구성 상세

  Dockerfile, extStartUp.sh, DataTracking 모듈 7개의 jar 파일

  docker-compose.yml, env.prod(스프링부트 설정파일), .env(DB 계정정보 설정파일)

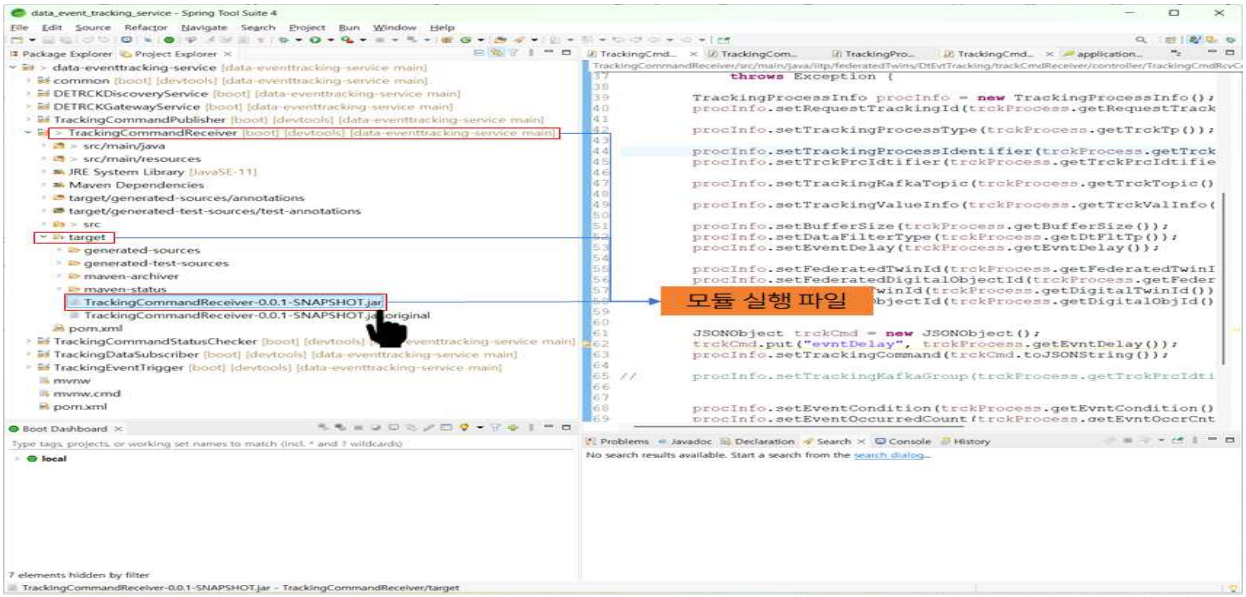
  db폴더 구조 : 파일(실행스크립트(create_table.sql), 데이터베이스 설정파일(my.conf))
- 다운받은 폴더를 서버에 업로드



5) 서버 주소 및 데이터 수정 파일

6) 실행 Jar파일 복사

가) 개발 내용이 수정되었을 경우 DataEventTrackingService 폴더의 .jar 파일을 Install 시 생성된 .jar 파일로 교체해야 함.



(a) Jar 파일 실행을 위한 쉘파일(extStartUp.sh)

- docker-compose.yml과 함께 각각의 포트 번호를 수정

```
extStartUp.shs - Windows 메모장
파일(F)  편집(E)  서식(O)  보기(V)  도움말(H)
#!/bin/bash

echo 'Starting DETRCKGatewayService-0.0.1-SNAPSHOT Server...'
java -XX:+UseParallelGC -XX:MaxGCPauseMillis=200 -Xms64m -Xmx128m -Djava.net.preferIPv4Stack=true -Duser.timezone=Asia/Seoul ₩
-jar DETRCKGatewayService.jar --server.port=8000 &
sleep 5

echo 'Starting TrackingCommandReceiver-0.0.1-SNAPSHOT Server...'
java -XX:+UseParallelGC -XX:MaxGCPauseMillis=200 -Xms64m -Xmx128m -Djava.net.preferIPv4Stack=true -Duser.timezone=Asia/Seoul ₩
-jar TrackingCommandReceiver.jar --server.port=22419 &
sleep 5

echo 'Starting TrackingCommandPublisher-0.0.1-SNAPSHOT Server...'
java -XX:+UseParallelGC -XX:MaxGCPauseMillis=200 -Xms64m -Xmx128m -Djava.net.preferIPv4Stack=true -Duser.timezone=Asia/Seoul ₩
-jar TrackingCommandPublisher.jar --server.port=22429 &
sleep 5

echo 'Starting TrackingDataSubscriber-0.0.1-SNAPSHOT Server...'
java -XX:+UseParallelGC -XX:MaxGCPauseMillis=200 -Xms64m -Xmx128m -Djava.net.preferIPv4Stack=true -Duser.timezone=Asia/Seoul ₩
-jar TrackingDataSubscriber.jar --server.port=22439 &
sleep 5

echo 'Starting TrackingEventTrigger-0.0.1-SNAPSHOT Server...'
java -XX:+UseParallelGC -XX:MaxGCPauseMillis=200 -Xms64m -Xmx128m -Djava.net.preferIPv4Stack=true -Duser.timezone=Asia/Seoul ₩
-jar TrackingEventTrigger.jar --server.port=22449 &
sleep 5

echo 'Starting TrackingCommandStatusChecker-0.0.1-SNAPSHOT Server...'
java -XX:+UseParallelGC -XX:MaxGCPauseMillis=200 -Xms64m -Xmx128m -Djava.net.preferIPv4Stack=true -Duser.timezone=Asia/Seoul ₩
-jar TrackingCommandStatusChecker.jar & sleep 5

echo 'Starting DETRCKDiscoveryService-0.0.1-SNAPSHOT Server...'
java -XX:+UseParallelGC -XX:MaxGCPauseMillis=200 -Xms64m -Xmx128m -Djava.net.preferIPv4Stack=true -Duser.timezone=Asia/Seoul ₩
-jar DETRCKDiscoveryService.jar --server.port=8761
```

(b) Docker image 생성을 위한 Dockerfile

```
FROM adoptopenjdk/openjdk11:latest
ARG JAR_FILE1=DETRCKDiscoveryService-0.0.1-SNAPSHOT.jar
ARG JAR_FILE2=DETRCKGatewayService-0.0.1-SNAPSHOT.jar
ARG JAR_FILE3=TrackingCommandReceiver-0.0.1-SNAPSHOT.jar
ARG JAR_FILE4=TrackingCommandPublisher-0.0.1-SNAPSHOT.jar
ARG JAR_FILE5=TrackingDataSubscriber-0.0.1-SNAPSHOT.jar
ARG JAR_FILE6=TrackingEventTrigger-0.0.1-SNAPSHOT.jar
ARG JAR_FILE7=TrackingCommandStatusChecker-0.0.1-SNAPSHOT.jar
COPY ${JAR_FILE1} DETRCKDiscoveryService.jar
COPY ${JAR_FILE2} DETRCKGatewayService.jar
COPY ${JAR_FILE3} TrackingCommandReceiver.jar
COPY ${JAR_FILE4} TrackingCommandPublisher.jar
COPY ${JAR_FILE5} TrackingDataSubscriber.jar
COPY ${JAR_FILE6} TrackingEventTrigger.jar
COPY ${JAR_FILE7} TrackingCommandStatusChecker.jar
ADD extStartUp.sh /
RUN chmod +x /extStartUp.sh
CMD ["/extStartUp.sh"]
```

(c) DB 생성을 위한 .env

- mariaDB 계정 정보




```
MYSQL_HOST=localhost
MYSQL_PORT=3306
MYSQL_ROOT_PASSWORD={rootPw}
MYSQL_DATABASE=dataEventTrackings
MYSQL_USER={userId}
MYSQL_PASSWORD={userPw}
```

(d) application.properties 설정 값을 변경하기 위한 .env.prod

- 표시된 부분에 정보 입력

i. 빨강 표시 : 운용 서버 ip/port 입력

ii. 파랑 표시 : mariaDB ID, Pw 와 eureka ID, Pw 입력

iii. 초록 표시 : Kafka 서버 ip 입력

```
spring.datasource.hikari.jdbc-url=jdbc:mysql://{serverIp}:{dbPort}/dataEventTrackings?
autoCommit=false&useUnicode=true&characterEncoding=utf8
&useSSL=false&serverTimezone=UTC&autoReconnect=true
spring.datasource.hikari.username={userId}
spring.datasource.hikari.password={userPw}
spring.security.user.name={eurekaId}
spring.security.user.password={eurekaPw}
eureka.client.serviceUrl.defaultZone=http://{eurekaId}:{eurekaPw}@{serverIp}:{eurekaPort}/eureka/

spring.rabbitmq.host=rabbitmq_detrck
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
rabbitmq.exchange.name=federate.tracking

spring.kafka.bootstrap-servers={kafkaIp}:9092

script.engine.name=nashorn
```

7) 터미널 접속 후 4번에서 만든 폴더 위치로 이동 및 폴더 내 파일 리스트 확인

- 명령어 ll 또는 ls 입력

```
[root@localhost ~]# cd /tracking/DataEventTrackingService/
[root@localhost DataEventTrackingService]# ll
합계 372804
-rw-r--r-- 1 root root 57002361  5월 21 13:46 DETRCKDiscoveryService-0.0.1-SNAPSHOT.jar
-rw-r--r-- 1 root root 50025342  5월 21 13:46 DETRCKGatewayService-0.0.1-SNAPSHOT.jar
-rw-r--r-- 1 root root      822  5월 21 13:45 Dockerfile
-rw-r--r-- 1 root root 45163473  5월 21 13:46 TrackingCommandPublisher-0.0.1-SNAPSHOT.jar
-rw-r--r-- 1 root root 71498457  5월 21 13:46 TrackingCommandReceiver-0.0.1-SNAPSHOT.jar
-rw-r--r-- 1 root root 45146220  5월 21 13:46 TrackingCommandStatusChecker-0.0.1-SNAPSHOT.jar
-rw-r--r-- 1 root root 82899082  5월 21 13:46 TrackingDataSubscriber-0.0.1-SNAPSHOT.jar
-rw-r--r-- 1 root root 29986795  5월 21 13:46 TrackingEventTrigger-0.0.1-SNAPSHOT.jar
drwxr-xr-x 5 root root       48  5월 21 13:46 db
-rw-r--r-- 1 root root     1914  5월 21 13:46 docker-compose.yml
-rw-r--r-- 1 root root     1786  5월 21 13:46 extStartUp.sh
```

8) Dockerfile command로 build

(a) docker build -t msa-detrck .(이미지 생성 명령어)

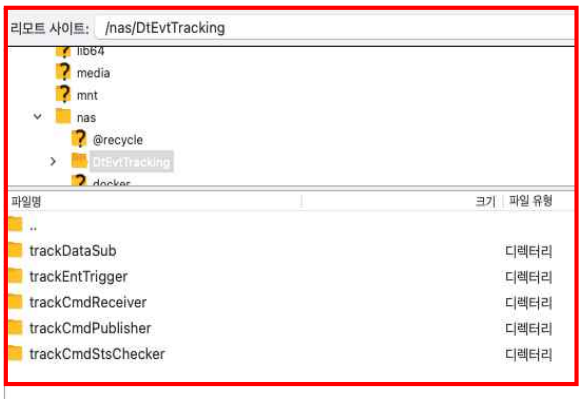
```
[root@localhost DataEventTrackingService]# docker build -t msa-detrck .
[+] Building 21.4s (15/15) FINISHED                                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                                            0.1s
 => => transferring dockerfile: 861B                                                                                            0.0s
 => [internal] load metadata for docker.io/adoptopenjdk/openjdk11:latest                                                        2.4s
 => [internal] load .dockerignore                                                                                               0.0s
 => => transferring context: 2B                                                                                                 0.0s
 => [internal] load build context                                                                                               4.7s
 => => transferring context: 381.81MB                                                                                           4.6s
 => [ 1/10] FROM docker.io/adoptopenjdk/openjdk11:latest@sha256:f73d2b5a5ad0c10744371b43dac5fb15440f434f6d998f4f97  16.2s
 => => resolve docker.io/adoptopenjdk/openjdk11:latest@sha256:f73d2b5a5ad0c10744371b43dac5fb15440f434f6d998f4f97340  0.0s
 => => sha256:835cfebbe9fda75f4911023815ac8db50ab7c542c407664cad3f913c55df45ed 4.52kB / 4.52kB                                  0.0s
 => => sha256:13b7e930469f6d3575a320709035c6acf6f5485a76abcf03d1b92a64c09c2476 27.51MB / 27.51MB                                2.6s
 => => sha256:d88cf8c01eff59496766a6ba94241177827c0a5f06687e1895ce1fa198d8fd7d 16.08MB / 16.08MB                                1.8s
 => => sha256:9d7ed34fbc01b8db77a208c71c64b973face030c29c93f321c116f08bf14c17d 196.25MB / 196.25MB                              2.8s
 => => sha256:f73d2b5a5ad0c10744371b43dac5fb15440f434f6d998f4f97340dc2fec036ac 1.72kB / 1.72kB                                  0.0s
 => => sha256:2fd1da64f50bfad26866b9225f92601df6177e5d9112d09ea8fe1efda189a3e5 954B / 954B                                      0.0s
 => => extracting sha256:13b7e930469f6d3575a320709035c6acf6f5485a76abcf03d1b92a64c09c2476                                       2.7s
 => => extracting sha256:d88cf8c01eff59496766a6ba94241177827c0a5f06687e1895ce1fa198d8fd7d                                       1.7s
 => => extracting sha256:9d7ed34fbc01b8db77a208c71c64b973face030c29c93f321c116f08bf14c17d                                       8.8s
 => [ 2/10] COPY DETRCKDiscoveryService-0.0.1-SNAPSHOT.jar DETRCKDiscoveryService.jar                                           0.6s
 => [ 3/10] COPY DETRCKGatewayService-0.0.1-SNAPSHOT.jar DETRCKGatewayService.jar                                               0.3s
 => [ 4/10] COPY TrackingCommandReceiver-0.0.1-SNAPSHOT.jar TrackingCommandReceiver.jar                                         0.2s
 => [ 5/10] COPY TrackingCommandPublisher-0.0.1-SNAPSHOT.jar TrackingCommandPublisher.jar                                       0.1s
 => [ 6/10] COPY TrackingDataSubscriber-0.0.1-SNAPSHOT.jar TrackingDataSubscriber.jar                                           0.2s
 => [ 7/10] COPY TrackingEventTrigger-0.0.1-SNAPSHOT.jar TrackingEventTrigger.jar                                               0.1s
 => [ 8/10] COPY TrackingCommandStatusChecker-0.0.1-SNAPSHOT.jar TrackingCommandStatusChecker.jar                               0.1s
 => [ 9/10] ADD extStartUp.sh /                                                                                                 0.0s
 => [10/10] RUN chmod +x /extStartUp.sh                                                                                         0.3s
 => exporting to image                                                                                                          0.8s
 => => exporting layers                                                                                                         0.8s
 => => writing image sha256:85981afabc156454871a4c6d6d9f6ec9a0a7e278ff2f0ba44da6102c2578a411                                    0.0s
 => => naming to docker.io/library/msa-detrck                                                                                   0.0s
```

(b) 이미지 리스트 확인 - 명령어 dokcer image ls

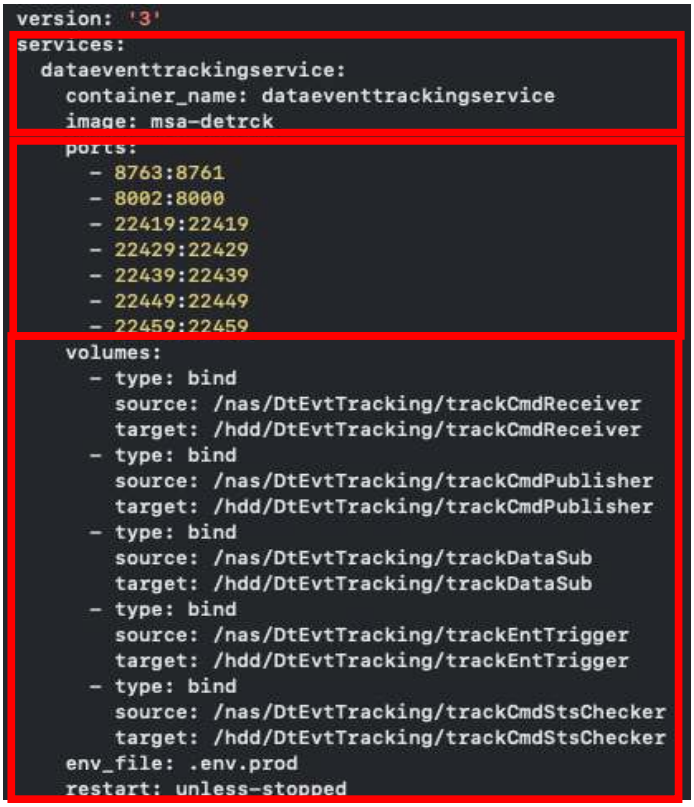
```
[root@localhost DataEventTrackingService]# docker image ls
REPOSITORY      TAG                       IMAGE ID       CREATED          SIZE
msa-detrck      latest                    85981afabc15   23 seconds ago   823MB
kazutod/ol      lo                        49e151952b21   4 months ago     1.1GB
alpine          latest                    b0c9d60fc5e3   4 months ago     7.83MB
kazutod/ini     kaz                       8b384b65da62   4 months ago     98.1MB
msa-fag         latest                    ce75de3c0258   7 months ago     757MB
msa-orc         latest                    69b22509f7d9   7 months ago     957MB
msa-ptz2        latest                    f935f79a7756   9 months ago     471MB
msa-ptz1        latest                    2770dbb17af2   9 months ago     471MB
mariadb         10                        fccf0a9f80df   11 months ago    404MB
rabbitmq        3.8.25-management-alpine  6cdbbce693fd   3 years ago      174MB
rabbitmq        3.7.14-management-alpine  eb91fc4abe55   6 years ago      132MB
```

9) Docker container 생성을 위한 docker-compose.yml 수정

- Port Forwarding 되는 외부 접속 포트는 방화벽 설정을 통하여 포트가 접속 가능한 상태로 변경하시기 바랍니다.



⇨ volumes의 모든 경로는 실제 서버에 파일을 생성한 후 진행



```
version: '3'
services:
  dataeventtrackingservice:
    container_name: dataeventtrackingservice
    image: msa-detrck
    ports:
      - 8763:8761
      - 8002:8000
      - 22419:22419
      - 22429:22429
      - 22439:22439
      - 22449:22449
      - 22459:22459
    volumes:
      - type: bind
        source: /nas/DtEvtTracking/trackCmdReceiver
        target: /hdd/DtEvtTracking/trackCmdReceiver
      - type: bind
        source: /nas/DtEvtTracking/trackCmdPublisher
        target: /hdd/DtEvtTracking/trackCmdPublisher
      - type: bind
        source: /nas/DtEvtTracking/trackDataSub
        target: /hdd/DtEvtTracking/trackDataSub
      - type: bind
        source: /nas/DtEvtTracking/trackEntTrigger
        target: /hdd/DtEvtTracking/trackEntTrigger
      - type: bind
        source: /nas/DtEvtTracking/trackCmdStsChecker
        target: /hdd/DtEvtTracking/trackCmdStsChecker
    env_file: .env.prod
    restart: unless-stopped
```

⇨ 컨테이너 및 빌드한 이미지명 작성

⇨ Port Forwarding

⇨ Mount 정보 및 설정파일 세팅



```
rabbitmq:
  container_name: rabbitmq_detrck
  image: rabbitmq:3.7.14-management-alpine
  volumes:
    - /hdd/detrck/rabbitmq/data/:/var/lib/rabbitmq/mnesia
    - /hdd/detrck/rabbitmq/logs/:/var/log/rabbitmq/
  environment:
    - RABBITMQ_DEFAULT_USER=admin
    - RABBITMQ_DEFAULT_PASS=admin
    - log.file=rabbit.log
    - log.file=true
    - log.file.level=info
    - log.console=true
    - log.console.level=info
  ports:
    - "5674:5672"
    - "15674:15672"
    - "25674:25672"
    - "4371:4369"
    - "5669:5671"
    - "15669:15671"
    - "1886:1883"
    - "8885:8883"
    - "61617:61613"
    - "61618:61614"
mariadb:
  image: mariadb:10
  ports:
    - 21219:3306
  volumes:
    - ./db/conf.d:/etc/mysql/conf.d
    - /hdd/dataTest/mariadb3:/hdd/data/mariadb
    - ./db/initdb.d:/docker-entrypoint-initdb.d
  env_file: .env
  environment:
    TZ: Asia/Seoul
  restart: unless-stopped
```

⇨ RabbitMq 정보 셋팅

⇨ MariaDB 정보 셋팅

10) 8번에서 생성한 docker-compose.yml 파일 command로 build

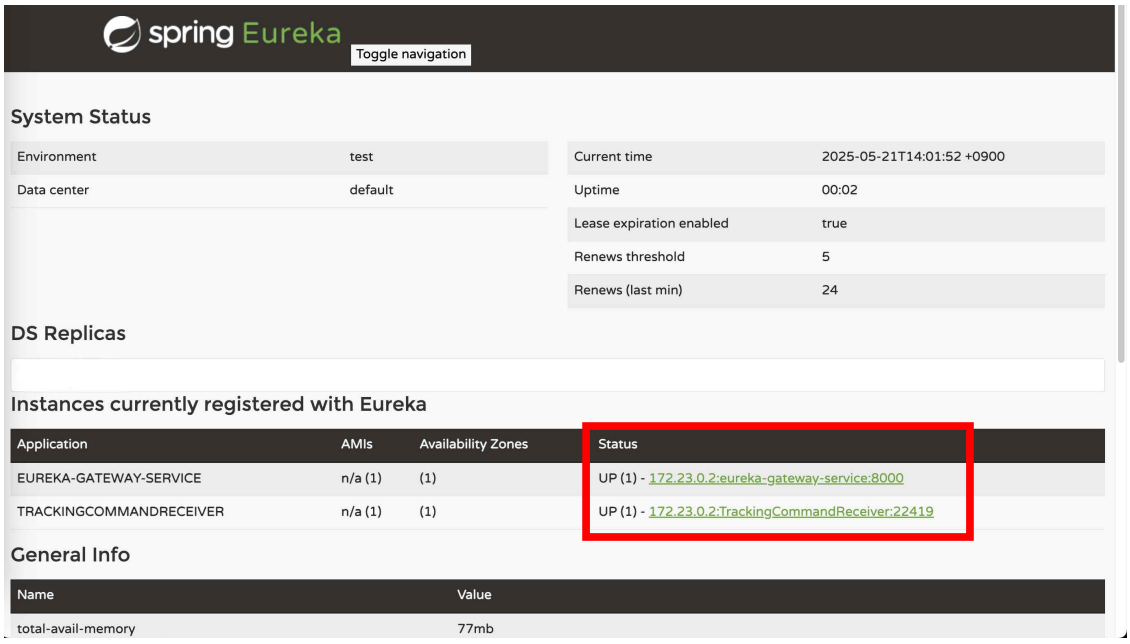
(a) dokcer-compose up -d(컨테이너 생성 명령어) 입력

```
[root@localhost DataEventTrackingService]# docker-compose up -d
Creating dataeventtrackingservice           ... done
Creating rabbitmq_detrck                    ... done
Creating dataeventtrackingservice_mariadb_1 ... done
```

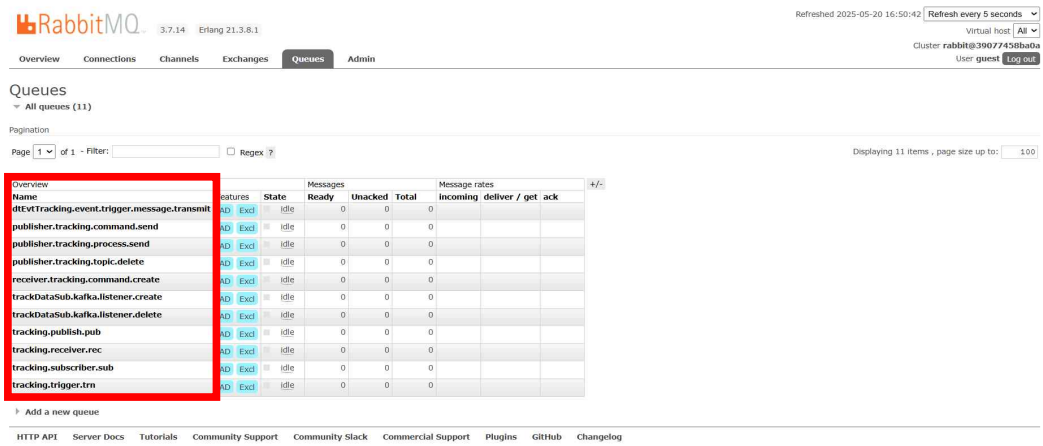
(b) 컨테이너 리스트 확인 - 명령어 docker ps -a

```
[root@localhost DataEventTrackingService]# docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED          STATUS          PORTS
                                                                                                                  NAMES
52ae2bb7f201   mariadb:10                           "docker-entrypoint.s…"   17 seconds ago   Up 16 seconds   0.0.0.
0:21219->3306/tcp, :::21219->3306/tcp
                                                                                                                  dataeventtracki
ngservice_mariadb_1
c84360c2f62c   rabbitmq:3.7.14-management-alpine    "docker-entrypoint.s…"   17 seconds ago   Up 16 seconds   0.0.0.
0:1886->1883/tcp, :::1886->1883/tcp, 0.0.0.0:4371->4369/tcp, :::4371->4369/tcp, 0.0.0.0:5669->5671/tcp, :::5669->56
71/tcp, 0.0.0.0:5674->5672/tcp, :::5674->5672/tcp, 0.0.0.0:8885->8883/tcp, :::8885->8883/tcp, 0.0.0.0:15669->15671/
tcp, :::15669->15671/tcp, 0.0.0.0:15674->15672/tcp, :::15674->15672/tcp, 0.0.0.0:25674->25672/tcp, :::25674->25672/
tcp, 0.0.0.0:61617->61613/tcp, :::61617->61613/tcp, 0.0.0.0:61618->61614/tcp, :::61618->61614/tcp   rabbitmq_detrck
39c9e20724ac   msa-detrck                           "/extStartUp.sh"         17 seconds ago   Up 16 seconds   0.0.0
.0:22419->22419/tcp, :::22419->22419/tcp, 0.0.0.0:22429->22429/tcp, :::22429->22429/tcp, 0.0.0.0:22439->22439/tcp,
:::22439->22439/tcp, 0.0.0.0:22449->22449/tcp, :::22449->22449/tcp, 0.0.0.0:22459->22459/tcp, :::22459->22459/tcp,
0.0.0.0:8002->8000/tcp, :::8002->8000/tcp, 0.0.0.0:8763->8761/tcp, :::8763->8761/tcp
                                                                                                                  dataeventtrack
ingservice
```

11) Euerka 서버 확인 => url: http://{serverIp}:{euerkaPort} (docker-compose.yml에서 설정한 포트)



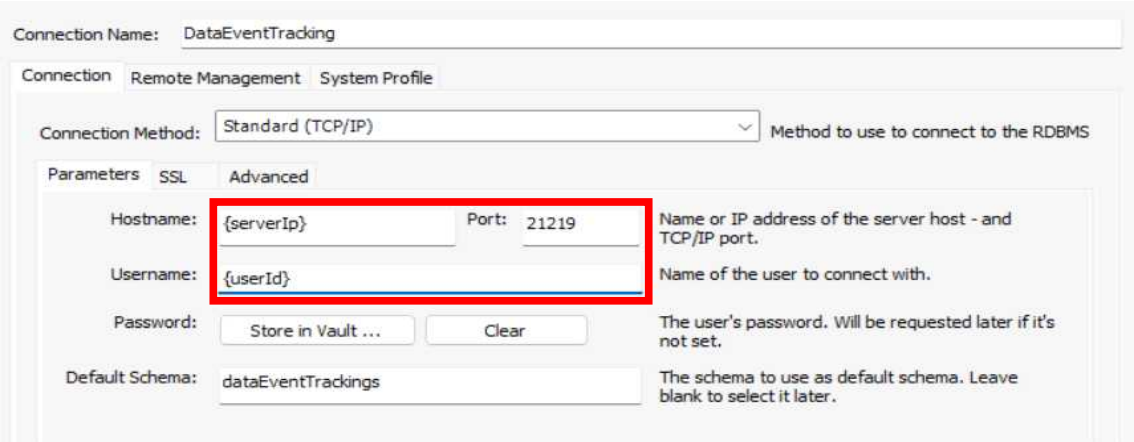
12) RabbitMq 서버 확인 => url: http://{serverIp}:{rabbitMqPort} (docker-compose.yml에서 설정한 포트)



13) MariaDB 확인

(a) MariaDB 연결 ex) MySQL WorkBench

- docker-compose.yml 파일에서 설정한 MariaDB 포트 입력
- env.prod에서 설정한 userId, pw입력

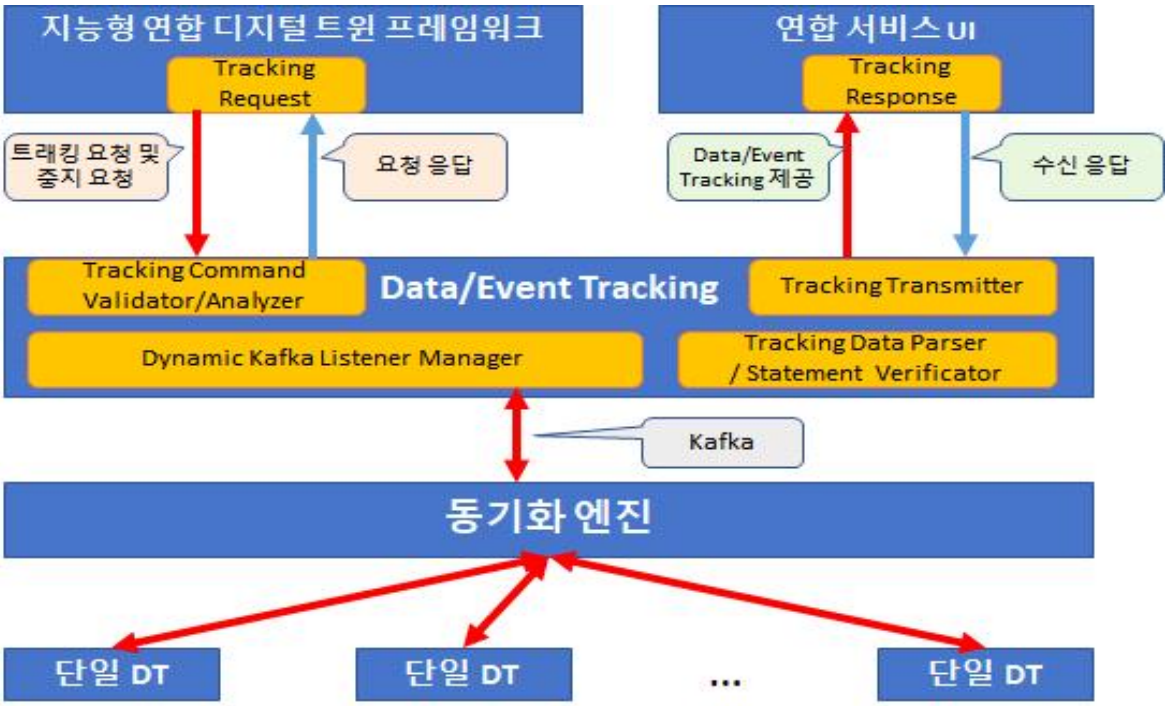


(b) DB 생성 확인



6. 동적 데이터/이벤트 트래킹 인터페이스

가. 인터페이스



< Data/Event Tracking 인터페이스 개념도 >

동적 데이터/이벤트 트래킹 인터페이스에는 지능형 연합 디지털트윈 프레임워크를 통하여 데이터/이벤트 트래킹 정보를 요청하는 트래킹 요청, 각 트래킹은 트래킹 요청 명령 중 evntOccrCnt(응답 횟수), evntEndDt(트래킹 구동 시간)설정 등의 조건으로 자동 종료됩니다. 하지만 사용자의 요청으로 트래킹을 요청 후 강제로 종료할 경우 사용하는 트래킹 중지 요청이 있습니다.

트래킹 요청 이후 트래킹 처리에 대한 데이터수신처리 기능(CallBack URL)을 위한 인터페이스를 제공합니다.

1) 트래킹 요청

가) 인터페이스 정의

(1) 트래킹 요청(Reqeust) 정의

Request Structure				
URI Path				
/fdt/detrck/rcv/tracking-commands				
Method : Post				Content-Type : application/json
Element	Type	M/O	Default Value	Description
reqIdentifier	string	M	-	**●트래킹 명령 요청 식별자** -트래킹 요청 시 단위 요청에 대한 식별자 -트래킹 이벤트 정보에 포함되어 전송
callBackURL	string	M	-	**●트래킹 이벤트 수신을 위한 응답 받을 URL** **●트래킹 서버에서 전송 (접속) 가능한 URL**
trckProcInfo []	JsonArray	M	-	**●데이터 /이벤트 트래킹 프로세스 정보** - 트래킹 프로세스 타입 , 데이터 트래킹 인터페이스 정보 (Topic, Group) 정보 , 데이터 주기 , 데이터 필터 타입 등 - 이벤트 발생 타입 및 이벤트 발생 /종료에 대한 정보 등의 요소로 구성 - 상세구조는 Data/Event Tracking Command 정의 참조
예제				
http ://{ServerIP}:{receiver service port}/fdt/detrck/rcv/tracking-commands				

(2) Request 요청 Tracking Command(Body) 예제


```
{
  "reqIdentifier": "RQ-987",
  "callBackURL": "{callback_url}",
  "trckProcInfo": [
    {
      "trckTp": "DTP",
      "trckPrcIdtifier": "DT-13",
      "federatedTwinId": "KR-02-K10000-20240000",
      "federatedDigitalObjectId": "F-KR-104111-0011",
      "digitalObjId": "KR-109941-0004",
      "trckTopic": "FDO_F-KR-104111-0011",
      "trckValInfo": "{₩"digitalObjectPath₩":₩"weatherMeasurement.list.wind_vvv₩",₩"objValType₩":₩"DOUBLE₩"}",
      "evntDelay": 30000,
      "bufferSize": 10,
      "dtFltTp": "DF01",
      "evntCondition": "{₩"checkable₩" : true}",
      "evntOccrCnt": -1,
      "evntEndDt": "{₩"INTERVAL₩" : ₩"DAYS₩", ₩"VALUE₩":3}"
    },
    {
      "trckTp": "DTP",
      "trckPrcIdtifier": "DT-23",
      "federatedTwinId": "KR-02-K10000-20240000",
      "federatedDigitalObjectId": "F-KR-104111-0011",
      "digitalObjId": "KR-109941-0004",
      "trckTopic": "FDO_F-KR-104111-0011",
      "trckValInfo": "{₩"digitalObjectPath₩":₩"weatherMeasurement.list.humidity₩",₩"objValType₩":₩"INTEGER₩"}",
      "evntDelay": 30000,
      "bufferSize": 5,
      "dtFltTp": "DF01",
      "evntCondition": "{₩"checkable₩" : true}",
      "evntOccrCnt": -1,
      "evntEndDt": "{₩"INTERVAL₩" : ₩"DAYS₩", ₩"VALUE₩":3}"
    },
    {
      "trckTp": "ETP",
      "trckPrcIdtifier": "EV-1323",
      "evntDelay": 30000,
      "bufferSize": 10,
      "evntCondition": "{₩"checkable₩": true,₩"type₩": ₩"CPW",₩"expression₩": {₩"ctrlStatement₩": ₩"((#{DT-13} > 1 && #{DT-13} < 10) || ( #{DT-23} > 90))₩"}}",
      "evntOccrCnt": -1,
      "evntEndDt": "{₩"INTERVAL₩" : ₩"DAYS₩", ₩"VALUE₩":3}"
    }
  ]
}
```

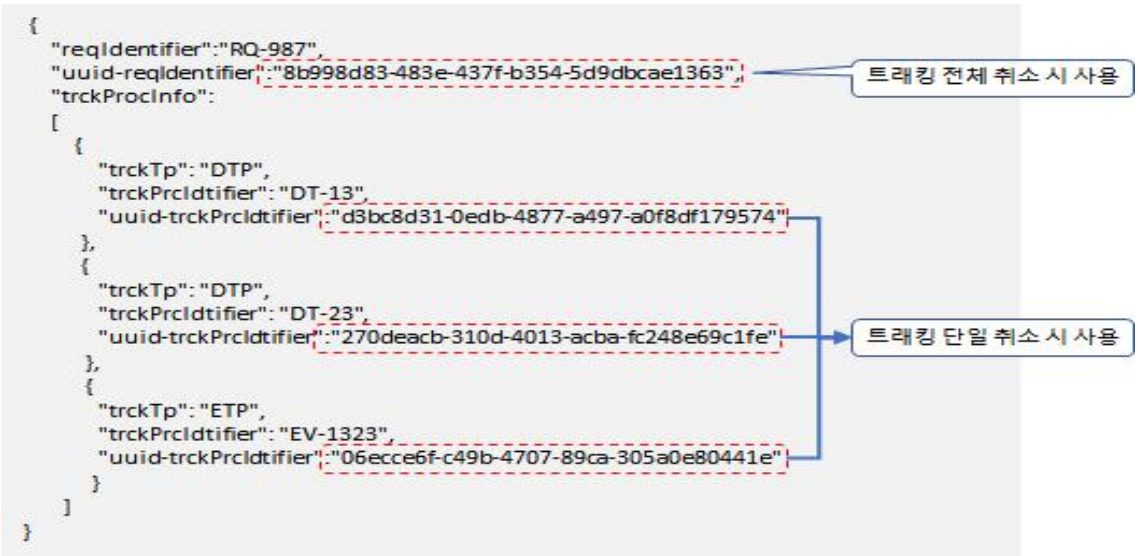
(3) 요청처리 결과(Response) 정의

<table>
<tr><th colspan="5">Response Structure</th></tr>
<tr><td colspan="5">Content-Type : application/json</td></tr>
<tr><th>Element</th><th>Type</th><th>M/O</th><th>Default Value</th><th>Description</th></tr>
<tr><td>reqIdentifier</td><td>string</td><td>M</td><td>-</td><td>●트래킹 명령 요청 식별자
-트래킹 요청 시 단위 요청에 대한 식별자</td></tr>
<tr><td>uuid-reqIdentifier</td><td>string</td><td>M</td><td>-</td><td>●서버에서 관리하는 트래킹 요청 식별자 정보
-트래킹 전체 취소 시 사용</td></tr>
<tr><td>trckProcInfo []</td><td>Json Array</td><td>M</td><td>-</td><td>●데이터 /이벤트 트래킹 프로세스 요청 결과 정보
- 트래킹 프로세스에 대한 서버의 관리 정보
<table>
<tr><th>Element</th><th>Type</th><th>Description</th></tr>
<tr><td>trckTp</td><td>String</td><td>- 프로세스 타입(요청정보)</td></tr>
<tr><td>trckPrcIdtifier</td><td>String</td><td>- 프로세스 식별자(요청정보)</td></tr>
<tr><td>uuid-trckPrcIdtifier</td><td>String</td><td>- 프로세스 별 서버 관리 식별자
- 트래킹 단위 취소 시 사용</td></tr>
</table>
</td></tr>
</table>

트래킹 요청 인터페이스 응답 코드

분류	HTTP Status	Response Code	메시지	비고
성공	Created	201	Created	장비 생성 요청 성공
실패	Bad request	400	Bad request	잘못된 요청
실패	Already exists	409	Already exists	이미 존재하는 장비정보
실패	Unprocessable Entity	422	Unprocessable Entity	생성 불가한 장비정보
실패	INTERNAL_SERVER_ERROR	500	Internal Server Error	서버처리 프로세스 오류

(4) 요청처리 결과 전송 예제



2) 트래킹 데이터 전송(트래킹 서버 -> 연합 서비스)

가) 인터페이스 정의

(1) 트래킹 데이터 전송

<table>
<tr><th colspan="5">Request Structure</th></tr>
<tr><th colspan="5">URI Path</th></tr>
<tr><td colspan="5">트래킹 요청 시 CallBack URL</td></tr>
<tr><td colspan="4">Method : Post</td><td>Content-Type : application/json</td></tr>
<tr><th>Element</th><th>Type</th><th>M/O</th><th>Default Value</th><th>Description</th></tr>
<tr><td>reqIdentifier</td><td>string</td><td>M</td><td>-</td><td>트래킹 요청 시 사용된 요청 식별자</td></tr>
<tr><td>trckTp</td><td>string</td><td>M</td><td>-</td><td>트래킹 이벤트 발생된 트래킹 타입</td></tr>
<tr><td>trckPrcIdtifier</td><td>string</td><td>M</td><td>-</td><td>트래킹 요청 시 사용된 프로세스 실행 시 단위 트래킹에 대한 식별자</td></tr>
<tr><td>evntOccuredTime</td><td>String</td><td>M</td><td>-</td><td>트래킹 발생 시각</td></tr>
<tr><td>trckValue</td><td>String</td><td>O</td><td>-</td><td>Tracking 발생 결과 값(Data Tracking 만 존재)</td></tr>
<tr><td>conditionResult</td><td>json</td><td>O</td><td>-</td><td>Tracking 발생 결과 값(Event Tracking 만 존재)
<table>
<tr><th>Element</th><th>Type</th><th>Description</th></tr>
<tr><td>ctrlDataValues</td><td>jsonArray</td><td>trckPrcIdtifier, trckValue 구성</td></tr>
<tr><td>evntCondition</td><td>boolean</td><td>- 이벤트 트래킹 처리 결과</td></tr>
</table>
</td></tr>
</table>

(2) 트래킹 데이터 전송(Body) 예제

(가) Data Tracking 전송 값 예제

```
{
  "reqIdentifier":"RQ-987",
  "trckTp":"DTP",
  "trckPrcIdtifier":"DT-13",
  "evntOccuredTime": "2024-11-08 13:00:03",
  "trckValue":"1.9604651162790696"
}
```

(나) Event Tracking 전송 값 예제


```
{
  “reqIdentifier”:”RQ-987”,
  “trckTp”:”ETP”,
  “trckPrcIdtifier”:” EV-1323”,
  "evntOccuredTime": "2024-11-08 13:00:03“,
  “conditionResult”:”
  {
     “ctrlDataValues”:
     [
       {
         "trckPrcIdtifier":"DT-13",
         "trckValue":"1.9604651162790696“
       },
       {
         "trckPrcIdtifier":"DT-23",
         "trckValue":"76“
       }
     ],
     "evntCondition":true
  }”
}
```


3) 트래킹 실행 중지(삭제) 요청

가) 인터페이스 정의

(1) 트래킹 요청(Request) 정의

Request Structure				
URI Path				
/fdt/detrck/rcv/tracking-commands/ {uuid-reqIdentifier}/trckProcInfos				
Method : Delete				Content-Type : application/json
Element	Type	M/O	Default Value	Description
uuid-reqIdentifier	string	M	-	PathVariable 로서 트래킹 요청 /응답 시 전송 받은 uuid-reqIdentifier 값 사용
Response				
• Http Status 정보 리턴 - HttpStatus.OK (200) 정상 - HttpStatus.INTERNAL_SERVER_ERROR(500) -> 서버 처리 오류				
예제				
http ://{ServerIP}:{receiver service port}/fdt/detrck/rcv/tracking-commands/8b998d83-483e-437f-b354-5d9dbcae1363/trckProcInfos				

7. Data/Event Tracking Command 정의

동적 데이터/이벤트 트래킹 인터페이스에서 사용하는 트래킹 요청을 구성하는 방법을 정의하고 동적인 트래킹 요청을 생성하여 원할한 서비스를 지속할 수 있는 방법을 제공하고자 합니다.

먼저 트래킹 요청은 다양한 트래킹 요청 구성을 위한 트래킹 프로세스를 정의하고 있으며 이러한 트래킹 프로세스를 사용자 설정정보 및 고속·고신뢰 양방향 동기화 엔진이 제공하는 연합/단일 DT의 메타정보 및 데이터 구조를 기반으로 하여 동적으로 구성하여 요청/서비스 받을 수 있습니다.

이러한 요청은 설정된 요청정보(evntOccrCnt(응답 횟수), evntEndDt(트래킹 구동 시간) 등)를 활용하여 자동으로 구동되며 다양한 형태의 DT 간 데이터를 효과적으로 통제하고 추적할 수 있습니다.

가. Data/Event Tracking Process Info 정의

<table>
<tr><th>Name</th><th>Type</th><th>Description</th><th>필수여부</th></tr>
<tr><td>trckTp</td><td>String</td><td>•트래킹 프로세스 타입 (타입 선택 정보)
- 데이터 트래킹 (1~N): DTP(Data Tracking Process)
- 이벤트 트래킹 (0-1): ETP(Event Tracking Process)</td><td>•필수</td></tr>
<tr><td>trckPrcIdtifier</td><td>String</td><td>•트래킹 프로세스 실행 시 단위 트래킹에 대한 식별자 (사용자 설정 정보)</td><td>•필수</td></tr>
<tr><td>federatedTwinId</td><td>String</td><td>•연합 디지털 트윈 ID (메타 데이터 등록 정보)</td><td>•데이터 트래킹 만 필수</td></tr>
<tr><td>federatedDigitalObjectId</td><td>String</td><td>•연합 디지털트윈 디지털객체 ID (메타 데이터 등록 정보)</td><td>•데이터 트래킹 만 필수</td></tr>
<tr><td>digitalObjectId</td><td>String</td><td>•연동 할 단일 디지털 객체 ID (메타 데이터 등록 정보)</td><td>•데이터 트래킹 만 필수</td></tr>
<tr><td>trckTopic</td><td>String</td><td>•트래킹 프로세스 별 인터페이스 할 Topic
(고속/고신뢰 동기화 엔진 연계 용 (Kafka)) -> (“FDO_” + {federatedDigitalObjectId })</td><td>•데이터 트래킹 만 필수</td></tr>
<tr><td>trckValInfo</td><td>json</td><td>•Data Tracking 처리 시 디지털트윈에서 트래킹 받을 값 (패스)
(메타정보 + 사용자 설정 정보)
- Json 구조로 연계되는 데이터를 동적으로 정제할 수 있는 JsonPath 기술 적용
- JsonPath는 Json객체의 표준화된 탐색 방법
<table><tr><th>Element</th><th>Type</th><th>Description</th></tr><tr><td>digitalObjectPath</td><td>String</td><td>- 트래킹 데이터 구조 (JsonPath)</td></tr><tr><td>objValType</td><td>String</td><td>- 트래킹 데이터 연계 타입(선택)
* 선택 설정 값 *
- INTEGER
- LONG
- FLOAT
- DOUBLE</td></tr></table>- 적용 예제
“trckValInfo“: “ { \“digitalObjectPath\“: \“trafficCongestion..list..tfvl\“, \“objValType\“: \“INTEGER \“ }“</td><td>•데이터 트래킹 만 필수</td></tr>
<tr><td>evntDelay</td><td>long</td><td>•Tracking 발생 주기 (밀리초 : millisecond, ms) (사용자 설정 정보)</td><td>•필수</td></tr>
<tr><td>bufferSize</td><td>int</td><td>•1 차 필터에 적용할 버퍼의 사이즈 (= 데이터 개수) (사용자 설정 정보)</td><td>•데이터 트래킹 만 필수</td></tr>
<tr><td>dtFltTp</td><td>String</td><td>•데이터 필터 타입 (타입 선택 정보)</td><td>-데이터 트</td></tr>
</table>

<table>
<tr><td></td><td></td><td>- DF01(min), DF02(max), DF03(average), DF04(sum) , DF05(stdevp) 등</td><td>래킹 만 필수</td></tr>
<tr><td>evntCondition</td><td>json</td><td>●이벤트 발생 조건 (사용자 설정 정보)
<table><tr><th>Element</th><th>Type</th><th>Description</th></tr><tr><td>checkable</td><td>boolean</td><td>true :
- 데이터 트래킹 : 필터 조건으로 생성된 데이터 전송
- 이벤트 트래킹 : 이벤트 제어 발생조건 체크 및 발생 데이터 전송
false : 이벤트 체크 /트래킹 데이터 전송 제외</td></tr><tr><td>expression</td><td>Json</td><td>- 이벤트 트래킹을 판단할 제어 구분
<table><tr><th>Element</th><th>Type</th><th>Description</th></tr><tr><td>ctrlStatement</td><td>json</td><td>- 이벤트 트래킹을 판단할 제어 구분
- 이벤트 트래킹 : #{trckPrcIdtifier} 는 체크해야 할 데이터 트래킹 식별자로 제어 결과
값이 치환할 변수 위치</td></tr></table></td></tr></table>
● 제어 예제
{
“checkable“ : true,
“expression“:
{
“ctrlStatement“:“((#{trckPrcIdtifier1} >100 &&#{trckPrcIdtifier1} <1000) ¦¦ (#{trckPrcIdtifier2} == “ON“))“
}
}</td><td>●필수</td></tr>
<tr><td>evntOccrCnt</td><td>int</td><td>●트래킹 이벤트 발생 정보 전송 건수 이후 트래킹 종료
(사용자 설정 정보)
-1 일 경우 미 적용
-default : 100</td><td></td><td></td></tr>
<tr><td>evntEndDt</td><td>json</td><td>●트래킹 이벤트 트래킹 시작 후 종료일 (DATE)까지 이벤트 전송
(사용자 설정 정보)
<table><tr><th>Element</th><th>Type</th><th>Description</th></tr><tr><td>INTERVAL</td><td>String</td><td>- 트래킹 종료 시간 주기(선택)
* 선택 설정 값 *
- SECONDS
- MINUTES
- HOURS
- DAYS</td></tr><tr><td>VALUE</td><td>int</td><td>- 트래킹 종료 시간 설정</td></tr><tr><td colspan="3">- default
{
“INTERVAL“ : “HOUR”,
“VALUE“ : 1
}</td></tr></table></td><td>- 미 적용일 경우 default 값 적용</td><td>-</td></tr>
</table>

나. Data Tracking 연계 값 설정(trckValInfo)

트래킹 요청 설정 시 단일트위에서 연계되는 값을 구조적으로 설정하는 방법을 설명하며 이 설명에 앞서 고속·고신뢰 양방향 동기화 엔진에서 제공하고 있는 연합/단일 DT의 메타 정보 및 데이터 구조를 확인하는 방법이 선행되어야 한다.

고속·고신뢰 양방향 동기화 엔진에서 제공하고 있는 연합 트윈 및 연합 트윈 객체 데이터 조회 방법(첨부파일 참조)을 통하여 JsonPath 구조를 확인할 수 있다.

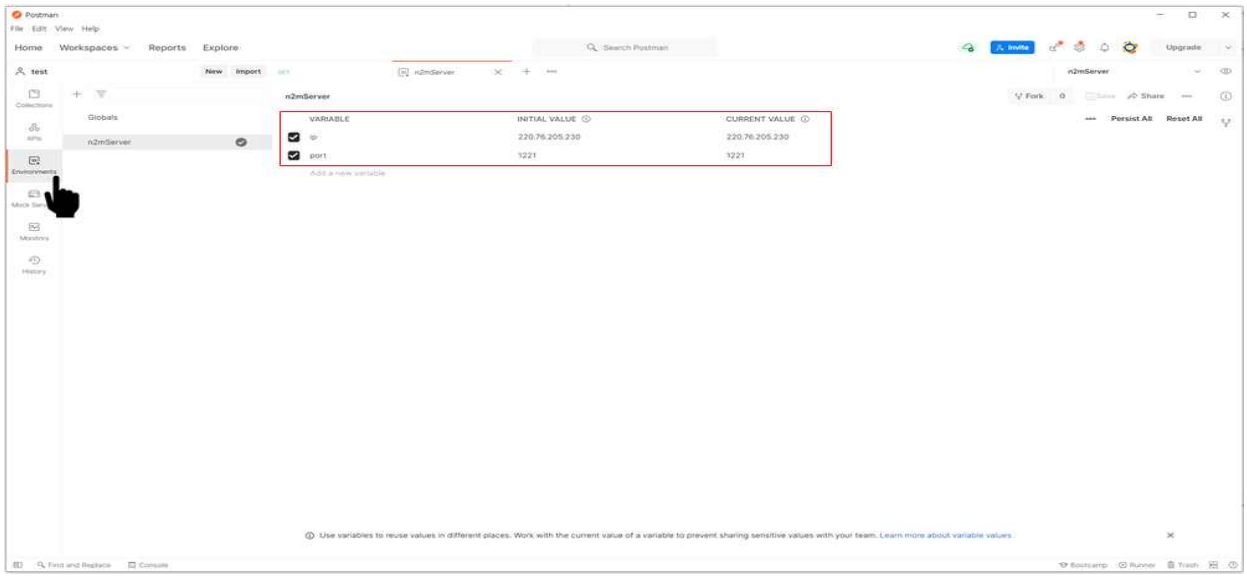
1) Git에서 /initResource/FedereatedTwin-n2m.postman_collection_.json 파일 다운로드

2) PostMan을 활용한 Meta정보 확인 방법

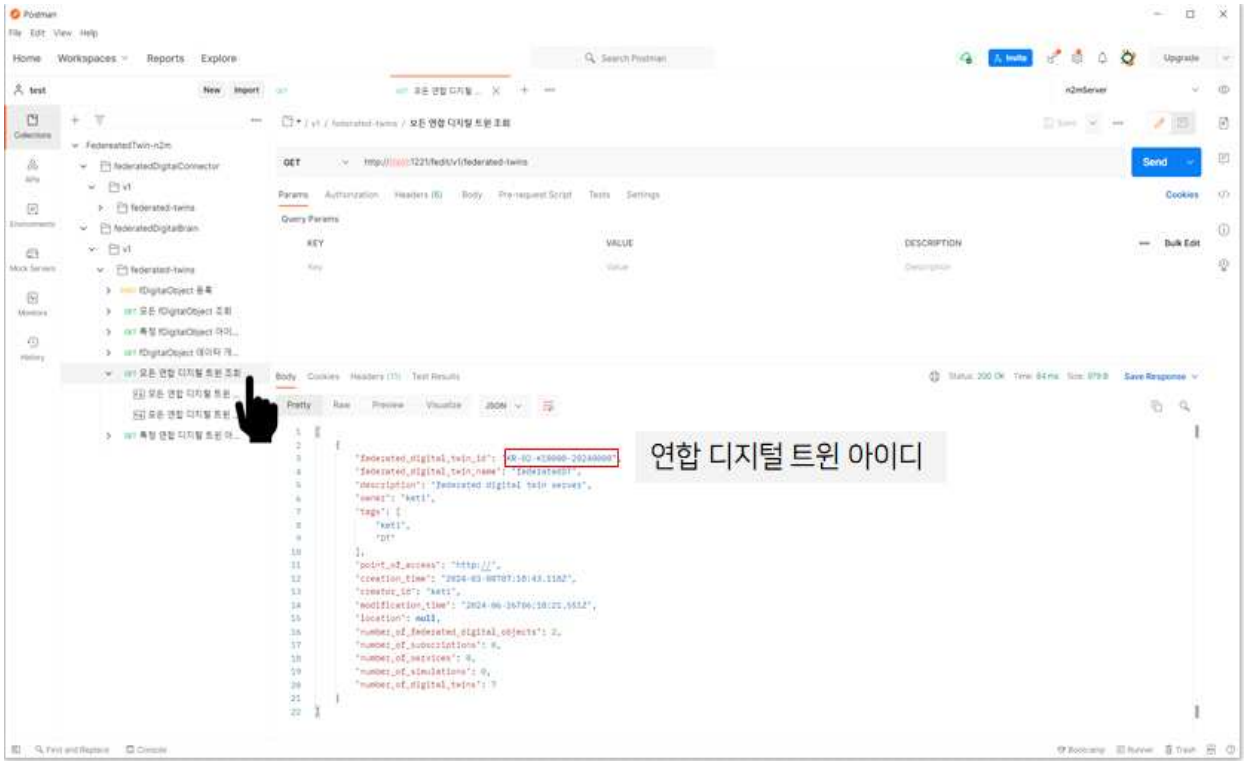
가) 정보 셋팅



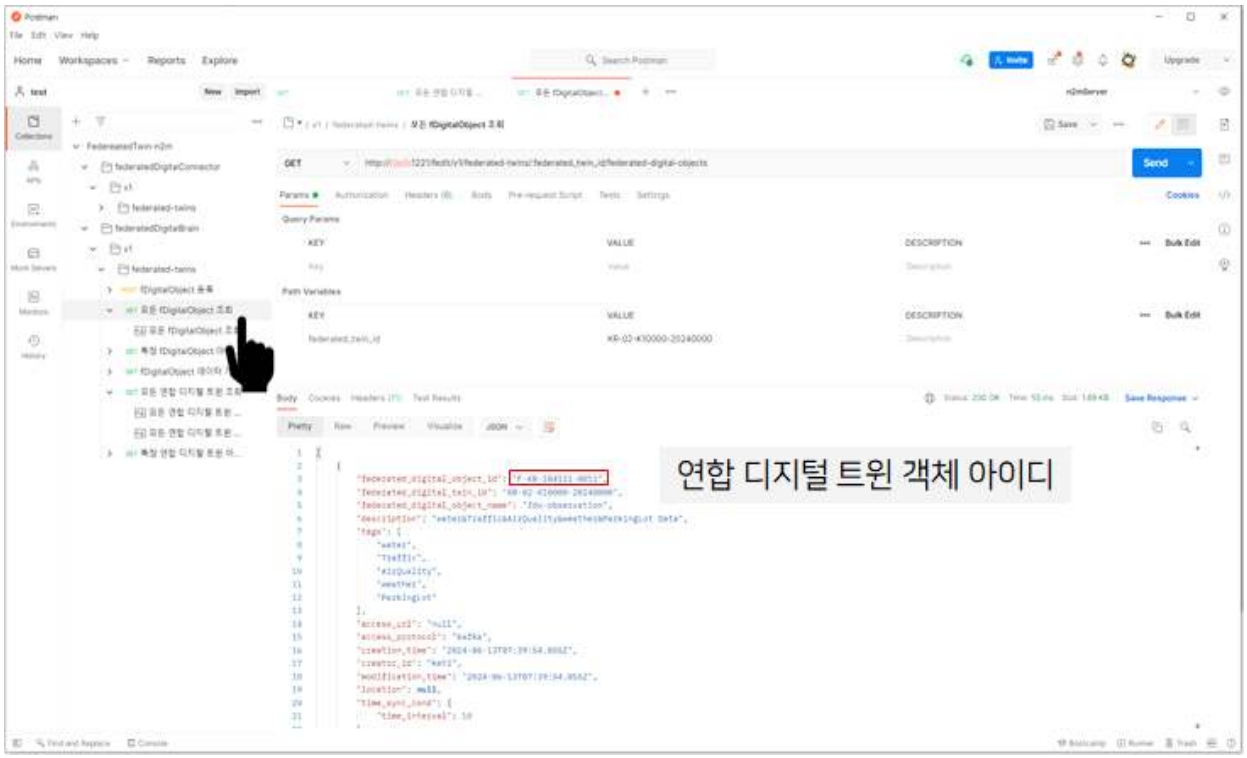
나) 파라메터 셋팅(서버주소, 환경설정)



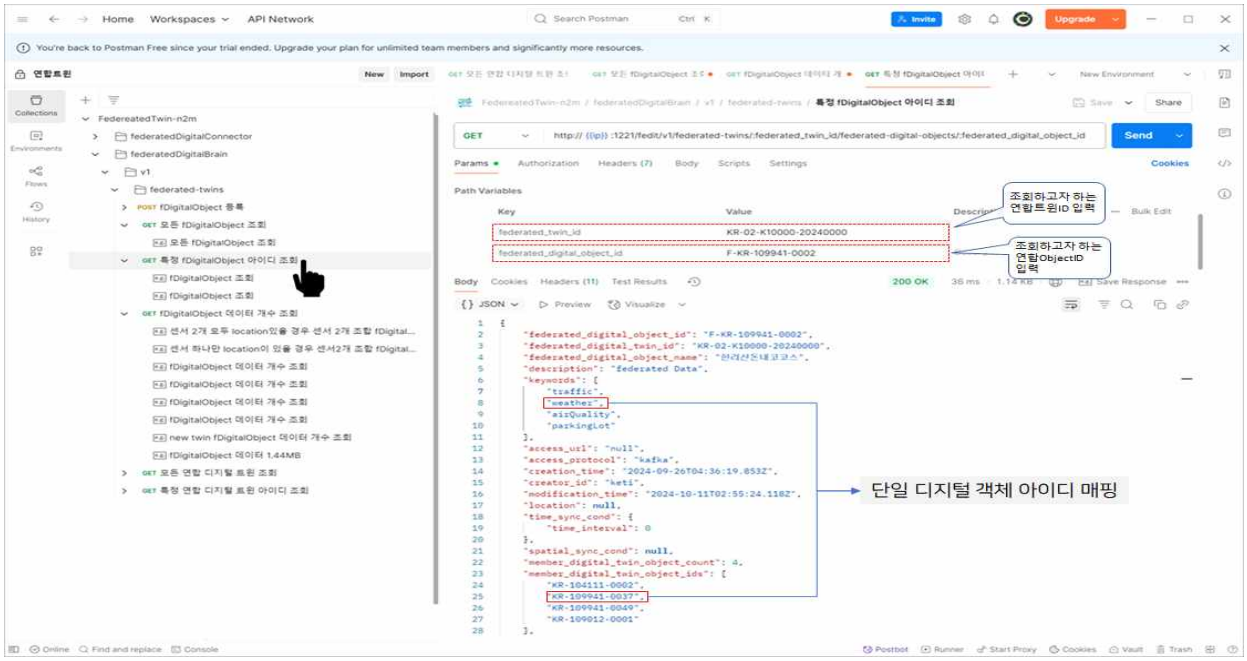
다) 등록된 연합트윈 정보 조회 (**federatedTwinId 설정**)



라) 등록된 연합 디지털 트윈 객체 조회(**federatedDigitalObjectId 설정**)

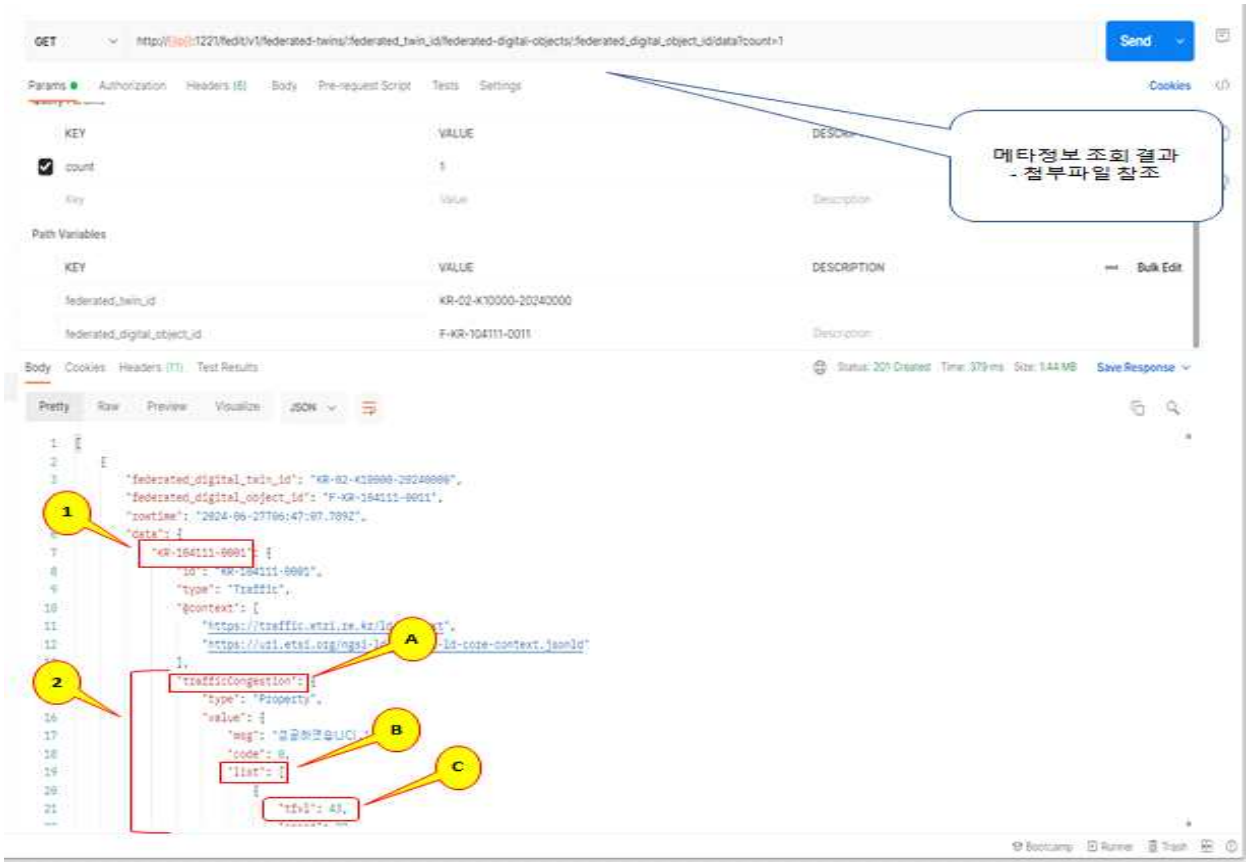


마) 등록된 단일 디지털 객체 조회(digitalObjectId)



바) 등록된 연합 디지털 트윈 객체 조회(trckValInfo)

- fDigitalObject 데이터 개수 조회 활용



연합트윈 메타정보에 대한 선택된 구조를 trckValInfo의 정보로 설정 예
"trckValInfo": "{\"digitalObjectPath\":\"trafficCongestion..list..tfvl\",\"objValType\":\"INT\"}"   A B C
1 자동으로 Path 설정 부분
2 정보를 확인하고 설정해야 하는 부분

다. 데이터/이벤트 트래킹 예시

